180+ Questions with Answers: The Ultimate C# Coding Interview Workbook: Algorithms, Data Structures & Problem Solving

String Manipulation

✅ 1. Reverse a string without using built-in functions

```
string ReverseString(string input)
{
    char[] reversed = new char[input.Length];
    for (int i = input.Length - 1, j = 0; i >= 0; i--, j++)
    {
        reversed[j] = input[i];
    }
    return new string(reversed);
}
```

Example:

```
Console.WriteLine(ReverseString("hello")); // Output: "olleh"
```

Explanation:
We loop through the string from the end and copy characters to a new array from the start.

✅ 2. Check if a string is a palindrome

```
bool IsPalindrome(string input)
{
    int left = 0, right = input.Length - 1;
    while (left < right)
    {
        if (input[left] != input[right])
            return false;
        left++;
        right--;
    }
    return true;
}
```

Example:

```
Console.WriteLine(IsPalindrome("madam")); // Output: True
Console.WriteLine(IsPalindrome("hello")); // Output: False
```

Explanation:
A palindrome reads the same backward. Compare characters from both ends moving inward.

✅ 3. Find the first non-repeated character

```
char? FirstNonRepeatedChar(string input)
{
    Dictionary<char, int> counts = new Dictionary<char, int>();

    foreach (char c in input)
    {
        if (counts.ContainsKey(c))
            counts[c]++;
        else
            counts[c] = 1;
    }

    foreach (char c in input)
```

```
    {
        if (counts[c] == 1)
            return c;
    }

    return null; // All characters are repeated
}
```

Example:

```
Console.WriteLine(FirstNonRepeatedChar("swiss")); // Output: 'w'
```

Explanation:
First pass: count frequency. Second pass: find the first char with count 1.

✅ 4. Count the number of vowels and consonants

```
void CountVowelsAndConsonants(string input, out int vowels, out int
consonants)
{
    vowels = consonants = 0;
    string lower = input.ToLower();

    foreach (char c in lower)
    {
        if (char.IsLetter(c))
        {
            if ("aeiou".Contains(c))
                vowels++;
            else
                consonants++;
        }
    }
}
```

Example:

```
CountVowelsAndConsonants("Hello World", out int v, out int c);
Console.WriteLine($"Vowels: {v}, Consonants: {c}"); // Output:
Vowels: 3, Consonants: 7
```

Explanation:
Check if each character is a letter, then classify as vowel or consonant.

✅ 5. Find the longest substring without repeating characters

```
int LongestUniqueSubstring(string input)
{
    Dictionary<char, int> seen = new Dictionary<char, int>();
    int maxLength = 0, start = 0;

    for (int end = 0; end < input.Length; end++)
    {
        char c = input[end];

        if (seen.ContainsKey(c) && seen[c] >= start)
            start = seen[c] + 1;

        seen[c] = end;
        maxLength = Math.Max(maxLength, end - start + 1);
    }

    return maxLength;
}
```

Example:

```
Console.WriteLine(LongestUniqueSubstring("abcabcbb")); // Output: 3
("abc")
```

Explanation:
Use sliding window and dictionary to track characters and their positions.

✅ 6. Check if two strings are anagrams

```
bool AreAnagrams(string str1, string str2)
{
    if (str1.Length != str2.Length)
        return false;

    int[] counts = new int[256];

    foreach (char c in str1)
        counts[c]++;

    foreach (char c in str2)
    {
        counts[c]--;
        if (counts[c] < 0)
            return false;
    }

    return true;
}
```

Example:

```
Console.WriteLine(AreAnagrams("listen", "silent")); // Output: True
```

Explanation:
Two strings are anagrams if they contain the same characters with same counts.

✅ 7. Reverse words in a string

```
string ReverseWords(string input)
{
    string[] words = input.Split(' ');
    Array.Reverse(words);
    return string.Join(" ", words);
}
```

Example:

```
Console.WriteLine(ReverseWords("Hello world from C#")); // Output:
"C# from world Hello"
```

Explanation:
Split the string by space, reverse the array, and join it back.

✅ 8. Remove duplicate characters from a string

```
string RemoveDuplicates(string input)
{
    HashSet<char> seen = new HashSet<char>();
    StringBuilder result = new StringBuilder();

    foreach (char c in input)
    {
        if (!seen.Contains(c))
        {
            seen.Add(c);
            result.Append(c);
        }
    }

    return result.ToString();
}
```

Example:

```
Console.WriteLine(RemoveDuplicates("programming")); // Output:
"progamin"
```

Explanation:
Use a set to keep track of seen characters, and only add unique ones.

✅ 9. Find all permutations of a string

```
void Permute(string prefix, string remaining)
{
    if (remaining.Length == 0)
    {
        Console.WriteLine(prefix);
        return;
    }

    for (int i = 0; i < remaining.Length; i++)
    {
        string newPrefix = prefix + remaining[i];
        string newRemaining = remaining.Substring(0, i) +
remaining.Substring(i + 1);
        Permute(newPrefix, newRemaining);
    }
}
```

Example:

```
Permute("", "abc");
// Output: abc, acb, bac, bca, cab, cba
```

Explanation:
Recursive backtracking: Fix one character, permute the rest.

✅ 10. Check if a string contains only digits

```
bool IsAllDigits(string input)
{
    foreach (char c in input)
    {
        if (!char.IsDigit(c))
            return false;
    }
    return true;
}
```

Example:

```
Console.WriteLine(IsAllDigits("123456")); // Output: True
Console.WriteLine(IsAllDigits("123a"));   // Output: False
```

Explanation:
Loop through characters, return false if any character is not a digit.

Arrays and Lists

✅ 1. Maximum Sum Subarray (Kadane's Algorithm)

```
int MaxSubArraySum(int[] nums)
{
    int maxCurrent = nums[0];
    int maxGlobal = nums[0];

    for (int i = 1; i < nums.Length; i++)
    {
        maxCurrent = Math.Max(nums[i], maxCurrent + nums[i]);
        maxGlobal = Math.Max(maxGlobal, maxCurrent);
    }

    return maxGlobal;
}
```

Example:

```
Console.WriteLine(MaxSubArraySum(new int[] { -2, 1, -3, 4, -1, 2, 1,
-5, 4 }));
// Output: 6 (subarray: [4, -1, 2, 1])
```

Explanation:
Kadane's Algorithm keeps track of the current max sum at each step.

✅ 2. Find the Kth Largest Element in an Unsorted Array

```
int FindKthLargest(int[] nums, int k)
{
    Array.Sort(nums);
    return nums[nums.Length - k];
}
```

Example:

```
Console.WriteLine(FindKthLargest(new int[] { 3, 2, 1, 5, 6, 4 },
2)); // Output: 5
```

Explanation:
Sort the array and get the k th largest from the end.

🔁 For large datasets, you could use a MinHeap (PriorityQueue).

✅ 3. Find the Missing Number from 1 to n

```
int FindMissingNumber(int[] nums)
{
    int n = nums.Length + 1;
    int expectedSum = n * (n + 1) / 2;
    int actualSum = nums.Sum();
    return expectedSum - actualSum;
}
```

Example:

```
Console.WriteLine(FindMissingNumber(new int[] { 1, 2, 4, 5 })); //
Output: 3
```

Explanation:
Use sum formula for 1 to n and subtract the actual sum.

✅ 4. Find the Intersection of Two Arrays

```
int[] ArrayIntersection(int[] nums1, int[] nums2)
{
    HashSet<int> set1 = new HashSet<int>(nums1);
    HashSet<int> result = new HashSet<int>();

    foreach (int num in nums2)
    {
        if (set1.Contains(num))
            result.Add(num);
    }

    return result.ToArray();
}
```

Example:

```
Console.WriteLine(string.Join(",", ArrayIntersection(
    new int[] { 1, 2, 2, 1 },
    new int[] { 2, 2 }
))); // Output: 2
```

Explanation:
Use a hash set to check presence efficiently.

✅ 5. Rotate an Array by K Steps

```
void RotateArray(int[] nums, int k)
{
    k %= nums.Length;
    Reverse(nums, 0, nums.Length - 1);
    Reverse(nums, 0, k - 1);
    Reverse(nums, k, nums.Length - 1);
}
```

```
void Reverse(int[] arr, int start, int end)
{
    while (start < end)
    {
        int temp = arr[start];
        arr[start] = arr[end];
        arr[end] = temp;
        start++;
        end--;
    }
}
```

Example:

```
int[] arr = { 1, 2, 3, 4, 5, 6, 7 };
RotateArray(arr, 3);
Console.WriteLine(string.Join(",", arr)); // Output: 5,6,7,1,2,3,4
```

Explanation:
Reverse the entire array, then reverse parts.

✅ 6. Find the Pair with the Maximum Sum

```
(int, int) MaxSumPair(int[] nums)
{
    Array.Sort(nums);
    return (nums[nums.Length - 2], nums[nums.Length - 1]);
}
```

Example:

```
var pair = MaxSumPair(new int[] { 1, 7, 3, 4 });
Console.WriteLine($"{pair.Item1}, {pair.Item2}"); // Output: 4, 7
```

Explanation:
The two largest elements will give the maximum sum.

✅ 7. Move All Zeroes to End (Maintain Order)

```
void MoveZeroesToEnd(int[] nums)
{
    int index = 0;

    foreach (int num in nums)
    {
        if (num != 0)
        {
            nums[index++] = num;
        }
    }

    while (index < nums.Length)
        nums[index++] = 0;
}
```

Example:

```
int[] arr = { 0, 1, 0, 3, 12 };
MoveZeroesToEnd(arr);
Console.WriteLine(string.Join(",", arr)); // Output: 1,3,12,0,0
```

Explanation:
Shift non-zero elements forward, then fill remaining with 0s.

✅ 8. Merge Two Sorted Arrays

```
int[] MergeSortedArrays(int[] arr1, int[] arr2)
{
    int i = 0, j = 0;
    List<int> result = new List<int>();
```

```
    while (i < arr1.Length && j < arr2.Length)
    {
        if (arr1[i] < arr2[j])
            result.Add(arr1[i++]);
        else
            result.Add(arr2[j++]);
    }

    while (i < arr1.Length)
        result.Add(arr1[i++]);

    while (j < arr2.Length)
        result.Add(arr2[j++]);

    return result.ToArray();
}
```

Example:

```
int[] merged = MergeSortedArrays(new int[] { 1, 3, 5 }, new int[] {
2, 4, 6 });
Console.WriteLine(string.Join(",", merged)); // Output: 1,2,3,4,5,6
```

Explanation:
Use two pointers to merge like merge sort.

✅ 9. Find the First Duplicate Element

```
int? FirstDuplicate(int[] nums)
{
    HashSet<int> seen = new HashSet<int>();

    foreach (int num in nums)
    {
        if (seen.Contains(num))
            return num;
```

```
            seen.Add(num);
        }

        return null;
    }
```

Example:

```
Console.WriteLine(FirstDuplicate(new int[] { 1, 2, 3, 4, 2, 5 }));
// Output: 2
```

Explanation:
Track seen elements using a hash set.

✅ 10. Find the Longest Increasing Subsequence (LIS)

```
int LongestIncreasingSubsequence(int[] nums)
{
    int n = nums.Length;
    int[] dp = new int[n];
    Array.Fill(dp, 1);

    for (int i = 1; i < n; i++)
    {
        for (int j = 0; j < i; j++)
        {
            if (nums[i] > nums[j])
                dp[i] = Math.Max(dp[i], dp[j] + 1);
        }
    }

    return dp.Max();
}
```

Example:

```
Console.WriteLine(LongestIncreasingSubsequence(new int[] { 10, 9, 2, 5, 3, 7, 101, 18 }));
// Output: 4 (LIS: 2,3,7,101)
```

Explanation:
Dynamic programming approach with `O(n^2)` complexity.

For `O(n log n)` LIS, binary search can be used (optional if needed).

Linked List

Definition of a Node (Singly Linked List)

```
public class ListNode
{
    public int val;
    public ListNode next;

    public ListNode(int val = 0, ListNode next = null)
    {
        this.val = val;
        this.next = next;
    }
}
```

✅ 1. Reverse a Linked List

```
ListNode ReverseList(ListNode head)
{
    ListNode prev = null;
    ListNode current = head;

    while (current != null)
    {
        ListNode nextNode = current.next;
```

```
        current.next = prev;
        prev = current;
        current = nextNode;
    }

    return prev;
}
```

Explanation:
Iteratively reverse the `next` pointers to flip the list.

✅ 2. Find the Middle Element of a Linked List

```
ListNode FindMiddle(ListNode head)
{
    ListNode slow = head, fast = head;

    while (fast != null && fast.next != null)
    {
        slow = slow.next;
        fast = fast.next.next;
    }

    return slow;
}
```

Explanation:
Use the "slow and fast pointer" technique — slow moves 1 step, fast moves 2.

✅ 3. Detect a Cycle in a Linked List

```
bool HasCycle(ListNode head)
{
    ListNode slow = head, fast = head;
```

```
    while (fast != null && fast.next != null)
    {
        slow = slow.next;
        fast = fast.next.next;

        if (slow == fast)
            return true;
    }

    return false;
}
```

Explanation:
If there's a cycle, fast and slow pointers will eventually meet.

✅ 4. Find the Nth Node from the End

```
ListNode NthFromEnd(ListNode head, int n)
{
    ListNode first = head, second = head;

    for (int i = 0; i < n; i++)
    {
        if (first == null) return null;
        first = first.next;
    }

    while (first != null)
    {
        first = first.next;
        second = second.next;
    }

    return second;
}
```

Explanation:
Advance one pointer by n, then move both until the first reaches the end.

✅ 5. Merge Two Sorted Linked Lists

```
ListNode MergeSortedLists(ListNode l1, ListNode l2)
{
    ListNode dummy = new ListNode(0);
    ListNode tail = dummy;

    while (l1 != null && l2 != null)
    {
        if (l1.val < l2.val)
        {
            tail.next = l1;
            l1 = l1.next;
        }
        else
        {
            tail.next = l2;
            l2 = l2.next;
        }
        tail = tail.next;
    }

    tail.next = l1 ?? l2;
    return dummy.next;
}
```


Explanation:
Use a dummy node and merge like in merge sort.

✅ 6. Remove Duplicate Nodes from a Linked List (Sorted)

```
ListNode RemoveDuplicates(ListNode head)
{
    ListNode current = head;

    while (current != null && current.next != null)
    {
        if (current.val == current.next.val)
            current.next = current.next.next;
        else
            current = current.next;
    }

    return head;
}
```

Explanation:
Check if adjacent values are equal; skip duplicates.

✅ 7. Flatten a Multi-Level Linked List

Assuming the node has a `child` pointer (for multilevel doubly linked lists):

```
public class MultiNode
{
    public int val;
    public MultiNode next;
    public MultiNode child;

    public MultiNode(int val = 0, MultiNode next = null, MultiNode
child = null)
    {
        this.val = val;
        this.next = next;
        this.child = child;
    }
}
```

```
MultiNode Flatten(MultiNode head)
{
    if (head == null) return head;

    Stack<MultiNode> stack = new Stack<MultiNode>();
    MultiNode current = head;

    while (current != null)
    {
        if (current.child != null)
        {
            if (current.next != null)
                stack.Push(current.next);

            current.next = current.child;
            current.child = null;
        }

        if (current.next == null && stack.Count > 0)
            current.next = stack.Pop();

        current = current.next;
    }

    return head;
}
```

Explanation:
Use a stack to remember `next` nodes while diving into children.

✅ 8. Find the Intersection Point of Two Linked Lists

```
ListNode GetIntersectionNode(ListNode headA, ListNode headB)
{
    ListNode a = headA, b = headB;

    while (a != b)
```

```
    {
        a = a == null ? headB : a.next;
        b = b == null ? headA : b.next;
    }

    return a;
}
```

Explanation:
When traversing both lists, switch head when reaching end; they'll align.

✅ 9. Detect Cycle and Return Starting Point

```
ListNode DetectCycleStart(ListNode head)
{
    ListNode slow = head, fast = head;

    while (fast != null && fast.next != null)
    {
        slow = slow.next;
        fast = fast.next.next;

        if (slow == fast)
        {
            ListNode entry = head;
            while (entry != slow)
            {
                entry = entry.next;
                slow = slow.next;
            }
            return entry;
        }
    }

    return null;
}
```

Explanation:
After detecting the cycle, move one pointer to the head and both by 1 step — they meet at cycle start.

✅ 10. Split a Linked List Into Two Halves

```
void SplitList(ListNode head, out ListNode firstHalf, out ListNode secondHalf)
{
    ListNode slow = head, fast = head, prev = null;

    while (fast != null && fast.next != null)
    {
        prev = slow;
        slow = slow.next;
        fast = fast.next.next;
    }

    firstHalf = head;
    secondHalf = slow;

    if (prev != null)
        prev.next = null; // Break the list
}
```

Explanation:
Use slow/fast pointers to find the midpoint, then break the list into two halves.

Stacks and Queues

Basic Node for Linked List (for Stack or Queue)

```
public class Node

{
```

```
    public int val;
    public Node next;

    public Node(int val)
    {
        this.val = val;
        this.next = null;
    }
}
```

✅ 1. Implement a Stack using Arrays

```
public class StackArray
{
    private int[] stack;
    private int top;
    private int capacity;

    public StackArray(int size)
    {
        capacity = size;
        stack = new int[capacity];
```

```
        top = -1;
    }

    public void Push(int val)
    {
        if (top == capacity - 1) throw new
InvalidOperationException("Stack Overflow");
        stack[++top] = val;
    }

    public int Pop()
    {
        if (top == -1) throw new InvalidOperationException("Stack
Underflow");
        return stack[top--];
    }

    public int Peek()
    {
        if (top == -1) throw new InvalidOperationException("Empty
Stack");
        return stack[top];
    }
```

```
    public bool IsEmpty() => top == -1;
}
```

✅ 2. Implement a Stack using Linked List

```
public class StackLinkedList
{
    private Node top;

    public void Push(int val)
    {
        Node newNode = new Node(val);
        newNode.next = top;
        top = newNode;
    }

    public int Pop()
    {
        if (top == null) throw new InvalidOperationException("Stack
Underflow");
        int val = top.val;
        top = top.next;
        return val;
```

```
    }

    public int Peek()
    {
        if (top == null) throw new InvalidOperationException("Empty
Stack");
        return top.val;
    }

    public bool IsEmpty() => top == null;
}
```

✅ 3. Implement a Queue Using Two Stacks

```
public class QueueUsingStacks
{
    private Stack<int> input = new Stack<int>();
    private Stack<int> output = new Stack<int>();

    public void Enqueue(int val)
    {
        input.Push(val);
    }
```

```
    public int Dequeue()
    {
        if (output.Count == 0)
        {
            while (input.Count > 0)
                output.Push(input.Pop());
        }

        if (output.Count == 0)
            throw new InvalidOperationException("Queue is empty");

        return output.Pop();
    }

    public bool IsEmpty() => input.Count == 0 && output.Count == 0;
}
```

✅ 4. Check for Balanced Parentheses in an Expression

```
bool IsBalanced(string expression)
{
```

```
    Stack<char> stack = new Stack<char>();

    foreach (char c in expression)
    {
        if (c == '(' || c == '{' || c == '[')
            stack.Push(c);
        else if (c == ')' || c == '}' || c == ']')
        {
            if (stack.Count == 0) return false;

            char top = stack.Pop();
            if ((c == ')' && top != '(') ||
                (c == '}' && top != '{') ||
                (c == ']' && top != '['))
                return false;
        }
    }

    return stack.Count == 0;
}
```

✅ 5. Evaluate a Postfix Expression

```
int EvaluatePostfix(string expr)
{
    Stack<int> stack = new Stack<int>();
    string[] tokens = expr.Split(' ');

    foreach (string token in tokens)
    {
        if (int.TryParse(token, out int num))
        {
            stack.Push(num);
        }
        else
        {
            int b = stack.Pop();
            int a = stack.Pop();
            switch (token)
            {
                case "+": stack.Push(a + b); break;
                case "-": stack.Push(a - b); break;
                case "*": stack.Push(a * b); break;
                case "/": stack.Push(a / b); break;
```

```
            }
        }
    }

    return stack.Pop();
}
```

Example:

```
Console.WriteLine(EvaluatePostfix("2 3 1 * + 9 -")); // Output: -4
```

✅ 6. Design a Stack with Minimum Function (`getMin`)

```
public class MinStack
{
    private Stack<int> mainStack = new Stack<int>();
    private Stack<int> minStack = new Stack<int>();

    public void Push(int val)
    {
        mainStack.Push(val);
        if (minStack.Count == 0 || val <= minStack.Peek())
            minStack.Push(val);
```

```
    }

    public int Pop()
    {
        int val = mainStack.Pop();
        if (val == minStack.Peek())
            minStack.Pop();
        return val;
    }

    public int GetMin()
    {
        return minStack.Peek();
    }
}
```

✅ 7. Implement a Circular Queue

```
public class CircularQueue
{
    private int[] data;
    private int front, rear, size, capacity;
```

```
public CircularQueue(int k)
{
    capacity = k;
    data = new int[capacity];
    front = 0;
    rear = -1;
    size = 0;
}

public bool Enqueue(int value)
{
    if (IsFull()) return false;

    rear = (rear + 1) % capacity;
    data[rear] = value;
    size++;
    return true;
}

public bool Dequeue()
{
```

```
        if (IsEmpty()) return false;

        front = (front + 1) % capacity;
        size--;
        return true;
    }

    public int Front() => IsEmpty() ? -1 : data[front];
    public int Rear() => IsEmpty() ? -1 : data[rear];

    public bool IsEmpty() => size == 0;
    public bool IsFull() => size == capacity;
}
```

✅ 8. Sort a Stack (Using Recursion)

```
void SortStack(Stack<int> stack)
{
    if (stack.Count > 0)
    {
        int top = stack.Pop();
        SortStack(stack);
```

```
        InsertInSortedOrder(stack, top);
    }
}

void InsertInSortedOrder(Stack<int> stack, int val)
{
    if (stack.Count == 0 || val > stack.Peek())
    {
        stack.Push(val);
        return;
    }

    int top = stack.Pop();
    InsertInSortedOrder(stack, val);
    stack.Push(top);
}
```

Example:

```
Stack<int> s = new Stack<int>(new int[] { 3, 1, 4, 2 });
SortStack(s);
```

✅ 9. Check if a Stack Can Be Popped in a Certain Order

```
bool IsValidPopOrder(int[] push, int[] pop)
{
    Stack<int> stack = new Stack<int>();
    int j = 0;

    foreach (int val in push)
    {
        stack.Push(val);
        while (stack.Count > 0 && stack.Peek() == pop[j])
        {
            stack.Pop();
            j++;
        }
    }

    return stack.Count == 0;
}
```

Example:

```
Console.WriteLine(IsValidPopOrder(new int[] {1,2,3,4,5}, new int[]
{4,5,3,2,1})); // True
```

✅ 10. Design a Stack with Push, Pop, and `getMin` Function (Duplicate of #6)

Already implemented above in **#6** as `MinStack`.

Trees

Base Tree Node Class

```
public class TreeNode
{
    public int val;
    public TreeNode left;
    public TreeNode right;

    public TreeNode(int val = 0, TreeNode left = null, TreeNode
right = null)
    {
        this.val = val;
        this.left = left;
        this.right = right;
    }
}
```

✅ 1. Find the Height of a Binary Tree

```
int TreeHeight(TreeNode root)
{
    if (root == null) return 0;
```

```
    return 1 + Math.Max(TreeHeight(root.left),
TreeHeight(root.right));
}
```

Explanation:
Height is the longest path from root to a leaf. Recursively find the max depth of left and right subtrees.

✅ 2. Check if a Binary Tree is Balanced

```
bool IsBalanced(TreeNode root)
{
    return CheckHeight(root) != -1;
}

int CheckHeight(TreeNode node)
{
    if (node == null) return 0;

    int left = CheckHeight(node.left);
    if (left == -1) return -1;

    int right = CheckHeight(node.right);
    if (right == -1) return -1;

    if (Math.Abs(left - right) > 1)
        return -1;

    return 1 + Math.Max(left, right);
}
```

Explanation:
A tree is balanced if the height difference between left and right subtrees is at most 1 at every node.

✅ 3. Find the Lowest Common Ancestor (LCA) in a BST

```
TreeNode LowestCommonAncestor(TreeNode root, TreeNode p, TreeNode q)
{
    if (root == null) return null;

    if (p.val < root.val && q.val < root.val)
        return LowestCommonAncestor(root.left, p, q);
    else if (p.val > root.val && q.val > root.val)
        return LowestCommonAncestor(root.right, p, q);
    else
        return root;
}
```

Explanation:
In a BST, the LCA is the first node where one node is on the left and the other is on the right.

✅ 4. Level-Order Traversal (Breadth-First Search)

```
List<List<int>> LevelOrder(TreeNode root)
{
    List<List<int>> result = new List<List<int>>();
    if (root == null) return result;

    Queue<TreeNode> queue = new Queue<TreeNode>();
    queue.Enqueue(root);

    while (queue.Count > 0)
    {
        int levelSize = queue.Count;
        List<int> currentLevel = new List<int>();

        for (int i = 0; i < levelSize; i++)
        {
            TreeNode node = queue.Dequeue();
```

```
            currentLevel.Add(node.val);

            if (node.left != null) queue.Enqueue(node.left);
            if (node.right != null) queue.Enqueue(node.right);
        }

        result.Add(currentLevel);
    }

    return result;
}
```

Explanation:
Use a queue to traverse nodes level by level.

✅ 5. In-order, Pre-order, and Post-order Traversal

```
void InOrder(TreeNode root)
{
    if (root == null) return;
    InOrder(root.left);
    Console.Write(root.val + " ");
    InOrder(root.right);
}

void PreOrder(TreeNode root)
{
    if (root == null) return;
    Console.Write(root.val + " ");
    PreOrder(root.left);
    PreOrder(root.right);
}

void PostOrder(TreeNode root)
{
    if (root == null) return;
    PostOrder(root.left);
```

```
    PostOrder(root.right);
    Console.Write(root.val + " ");
}
```

Explanation:

- InOrder: Left → Root → Right
- PreOrder: Root → Left → Right
- PostOrder: Left → Right → Root

✅ 6. Find the Diameter of a Binary Tree

```
int maxDiameter = 0;

int Diameter(TreeNode root)
{
    Depth(root);
    return maxDiameter;
}

int Depth(TreeNode node)
{
    if (node == null) return 0;

    int left = Depth(node.left);
    int right = Depth(node.right);

    maxDiameter = Math.Max(maxDiameter, left + right);
    return 1 + Math.Max(left, right);
}
```

Explanation:
Diameter is the longest path between any two nodes. It's the max of (left depth + right depth) at any node.

✅ 7. Check if a Binary Tree is a Binary Search Tree (BST)

```
bool IsBST(TreeNode root)
{
    return Validate(root, long.MinValue, long.MaxValue);
}

bool Validate(TreeNode node, long min, long max)
{
    if (node == null) return true;

    if (node.val <= min || node.val >= max) return false;

    return Validate(node.left, min, node.val) &&
           Validate(node.right, node.val, max);
}
```

Explanation:
Each node must be greater than all nodes in its left subtree and less than all in the right.

✅ 8. Convert BST to Sorted Doubly Linked List (In-place)

```
TreeNode prev = null;
TreeNode head = null;

TreeNode BSTToDLL(TreeNode root)
{
    Convert(root);
    return head;
}

void Convert(TreeNode node)
```

```
{
    if (node == null) return;

    Convert(node.left);

    if (prev != null)
    {
        prev.right = node;
        node.left = prev;
    }
    else
    {
        head = node; // First node becomes head
    }

    prev = node;

    Convert(node.right);
}
```

Explanation:
Perform in-order traversal and link nodes as a doubly linked list.

✅ 9. Find Maximum Path Sum in a Binary Tree

```
int maxPath = int.MinValue;

int MaxPathSum(TreeNode root)
{
    MaxGain(root);
    return maxPath;
}

int MaxGain(TreeNode node)
{
    if (node == null) return 0;
```

```
    int left = Math.Max(0, MaxGain(node.left));
    int right = Math.Max(0, MaxGain(node.right));

    int localMax = node.val + left + right;
    maxPath = Math.Max(maxPath, localMax);

    return node.val + Math.Max(left, right);
}
```

Explanation:
For each node, compute max gain from left and right. Store the global max path sum.

✅ 10. Invert a Binary Tree (Mirror Image)

```
TreeNode Invert(TreeNode root)
{
    if (root == null) return null;

    TreeNode temp = root.left;
    root.left = Invert(root.right);
    root.right = Invert(temp);

    return root;
}
```

Explanation:
Swap the left and right child recursively.

Graphs

🧱 Graph Representation

We'll use an **adjacency list** to represent graphs:

```
Dictionary<int, List<int>> graph = new Dictionary<int, List<int>>();
```

Or, for weighted graphs:

```
Dictionary<int, List<(int neighbor, int weight)>> graph = new();
```

✅ 1. Depth-First Search (DFS)

```
void DFS(Dictionary<int, List<int>> graph, int start, HashSet<int>
visited)
{
    if (visited.Contains(start)) return;

    Console.WriteLine(start);
    visited.Add(start);

    foreach (var neighbor in graph[start])
        DFS(graph, neighbor, visited);
}
```

Explanation:
Recursively visit each neighbor; track visited nodes to avoid cycles.

✅ 2. Breadth-First Search (BFS)

```
void BFS(Dictionary<int, List<int>> graph, int start)
{
    Queue<int> queue = new Queue<int>();
    HashSet<int> visited = new HashSet<int>();

    queue.Enqueue(start);
    visited.Add(start);

    while (queue.Count > 0)
    {
        int node = queue.Dequeue();
        Console.WriteLine(node);
```

```
        foreach (int neighbor in graph[node])
        {
            if (!visited.Contains(neighbor))
            {
                visited.Add(neighbor);
                queue.Enqueue(neighbor);
            }
        }
    }
}
```

Explanation:
Explore nodes level by level using a queue.

✅ 3. Detect Cycle in Directed Graph

```
bool HasCycleDirected(Dictionary<int, List<int>> graph)
{
    HashSet<int> visited = new HashSet<int>();
    HashSet<int> recursionStack = new HashSet<int>();

    foreach (var node in graph.Keys)
    {
        if (IsCyclic(node, visited, recursionStack, graph))
            return true;
    }

    return false;
}

bool IsCyclic(int node, HashSet<int> visited, HashSet<int> stack,
Dictionary<int, List<int>> graph)
{
    if (stack.Contains(node)) return true;
    if (visited.Contains(node)) return false;

    visited.Add(node);
```

```
    stack.Add(node);

    foreach (var neighbor in graph[node])
    {
        if (IsCyclic(neighbor, visited, stack, graph))
            return true;
    }

    stack.Remove(node);
    return false;
}
```

Explanation:
Use recursion stack to detect back-edges (which form cycles).

✅ 4. Shortest Path in Unweighted Graph (BFS)

```
Dictionary<int, int> ShortestPathUnweighted(Dictionary<int,
List<int>> graph, int start)
{
    Queue<int> queue = new();
    Dictionary<int, int> distance = new();

    foreach (var node in graph.Keys)
        distance[node] = int.MaxValue;

    distance[start] = 0;
    queue.Enqueue(start);

    while (queue.Count > 0)
    {
        int node = queue.Dequeue();

        foreach (var neighbor in graph[node])
        {
            if (distance[neighbor] == int.MaxValue)
            {
```

```
                distance[neighbor] = distance[node] + 1;
                queue.Enqueue(neighbor);
            }
        }
    }

    return distance;
}
```

✅ 5. Topological Sort (DAG)

```
List<int> TopologicalSort(Dictionary<int, List<int>> graph)
{
    HashSet<int> visited = new();
    Stack<int> stack = new();

    foreach (var node in graph.Keys)
        if (!visited.Contains(node))
            TopoDFS(node, visited, stack, graph);

    return stack.ToList();
}

void TopoDFS(int node, HashSet<int> visited, Stack<int> stack,
Dictionary<int, List<int>> graph)
{
    visited.Add(node);

    foreach (var neighbor in graph[node])
        if (!visited.Contains(neighbor))
            TopoDFS(neighbor, visited, stack, graph);

    stack.Push(node);
}
```

Explanation:
DFS + post-order stack gives topological order for DAGs.

✅ 6. Shortest Path in Weighted Graph (Dijkstra's Algorithm)

```
Dictionary<int, int> Dijkstra(Dictionary<int, List<(int to, int weight)>> graph, int start)
{
    var distance = new Dictionary<int, int>();
    var pq = new PriorityQueue<int, int>();

    foreach (var node in graph.Keys)
        distance[node] = int.MaxValue;

    distance[start] = 0;
    pq.Enqueue(start, 0);

    while (pq.Count > 0)
    {
        pq.TryDequeue(out int node, out int dist);

        foreach (var (neighbor, weight) in graph[node])
        {
            int newDist = dist + weight;
            if (newDist < distance[neighbor])
            {
                distance[neighbor] = newDist;
                pq.Enqueue(neighbor, newDist);
            }
        }
    }

    return distance;
}
```

Explanation:
Use a min-heap (priority queue) to always expand the closest node first.

✅ 7. Check if a Graph is Bipartite

```
bool IsBipartite(Dictionary<int, List<int>> graph)
{
    Dictionary<int, int> colors = new();

    foreach (var node in graph.Keys)
    {
        if (!colors.ContainsKey(node) && !DFSColor(node, 0, colors,
graph))
            return false;
    }

    return true;
}

bool DFSColor(int node, int color, Dictionary<int, int> colors,
Dictionary<int, List<int>> graph)
{
    if (colors.ContainsKey(node))
        return colors[node] == color;

    colors[node] = color;

    foreach (var neighbor in graph[node])
    {
        if (!DFSColor(neighbor, 1 - color, colors, graph))
            return false;
    }

    return true;
}
```

Explanation:
Alternate colors between neighbors; if conflict occurs, it's not bipartite.

✅ 8. Detect if a Graph is Connected

```
bool IsConnected(Dictionary<int, List<int>> graph)
{
    HashSet<int> visited = new();
    int start = graph.Keys.First();
    DFS(graph, start, visited);

    return visited.Count == graph.Count;
}
```

Explanation:
If all nodes are visited from a single DFS, graph is connected.

✅ 9. Strongly Connected Components (Kosaraju's Algorithm)

```
List<List<int>> KosarajuSCC(Dictionary<int, List<int>> graph)
{
    Stack<int> stack = new();
    HashSet<int> visited = new();

    // Step 1: Topological Sort
    foreach (var node in graph.Keys)
        if (!visited.Contains(node))
            FillOrder(node, visited, stack, graph);

    // Step 2: Transpose the graph
    var transposed = TransposeGraph(graph);

    // Step 3: DFS using reverse topological order
    visited.Clear();
    List<List<int>> sccs = new();

    while (stack.Count > 0)
    {
        int node = stack.Pop();
```

```
        if (!visited.Contains(node))
        {
            List<int> scc = new();
            DFSCollect(node, visited, transposed, scc);
            sccs.Add(scc);
        }
    }

    return sccs;
}

void FillOrder(int node, HashSet<int> visited, Stack<int> stack,
Dictionary<int, List<int>> graph)
{
    visited.Add(node);
    foreach (var neighbor in graph[node])
        if (!visited.Contains(neighbor))
            FillOrder(neighbor, visited, stack, graph);

    stack.Push(node);
}

Dictionary<int, List<int>> TransposeGraph(Dictionary<int, List<int>>
graph)
{
    var transposed = new Dictionary<int, List<int>>();

    foreach (var node in graph.Keys)
        transposed[node] = new List<int>();

    foreach (var node in graph.Keys)
        foreach (var neighbor in graph[node])
            transposed[neighbor].Add(node);

    return transposed;
}
```

```
void DFSCollect(int node, HashSet<int> visited, Dictionary<int,
List<int>> graph, List<int> scc)
{
    visited.Add(node);
    scc.Add(node);

    foreach (var neighbor in graph[node])
        if (!visited.Contains(neighbor))
            DFSCollect(neighbor, visited, graph, scc);
}
```

Explanation:
Kosaraju's algorithm involves two DFS passes: one for ordering, one for transposed graph.

✅ 10. Floyd-Warshall Algorithm (All Pairs Shortest Paths)

```
int[,] FloydWarshall(int[,] graph)
{
    int n = graph.GetLength(0);
    int[,] dist = (int[,])graph.Clone();

    for (int k = 0; k < n; k++)
    {
        for (int i = 0; i < n; i++)
        {
            for (int j = 0; j < n; j++)
            {
                if (dist[i, k] != int.MaxValue && dist[k, j] !=
int.MaxValue)
                    dist[i, j] = Math.Min(dist[i, j], dist[i, k] +
dist[k, j]);
            }
        }
    }
```

```
    return dist;
}
```

Graph Matrix Format:

- `graph[i, j] = weight`

Dynamic Programming

1. Find the nth Fibonacci Number

```
int Fibonacci(int n)
{
    if (n <= 1) return n;

    int a = 0, b = 1;
    for (int i = 2; i <= n; i++)
    {
        int temp = a + b;
        a = b;
        b = temp;
    }
    return b;
}
```

Explanation:
Iterative DP approach; each Fibonacci number is sum of two previous.

2. 0/1 Knapsack Problem

```
int Knapsack(int[] weights, int[] values, int W)
{
    int n = weights.Length;
    int[,] dp = new int[n + 1, W + 1];

    for (int i = 1; i <= n; i++)
    {
        for (int w = 1; w <= W; w++)
        {
            if (weights[i - 1] <= w)
                dp[i, w] = Math.Max(dp[i - 1, w], values[i - 1] +
dp[i - 1, w - weights[i - 1]]);
            else
                dp[i, w] = dp[i - 1, w];
        }
    }
    return dp[n, W];
}
```

Explanation:
Build table where `dp[i,w]` = max value using first i items and capacity w.

3. Longest Common Subsequence (LCS)

```
int LCS(string s1, string s2)
{
    int m = s1.Length, n = s2.Length;
    int[,] dp = new int[m + 1, n + 1];

    for (int i = 1; i <= m; i++)
    {
        for (int j = 1; j <= n; j++)
        {
            if (s1[i - 1] == s2[j - 1])
                dp[i, j] = dp[i - 1, j - 1] + 1;
            else
                dp[i, j] = Math.Max(dp[i - 1, j], dp[i, j - 1]);
        }
    }
    return dp[m, n];
}
```

Explanation:
Build DP table comparing chars; find longest subsequence common to both strings.

4. Longest Increasing Subsequence (LIS)

```
int LIS(int[] nums)
{
    int n = nums.Length;
    int[] dp = new int[n];
    Array.Fill(dp, 1);

    int maxLen = 1;

    for (int i = 1; i < n; i++)
    {
        for (int j = 0; j < i; j++)
        {
            if (nums[i] > nums[j])
                dp[i] = Math.Max(dp[i], dp[j] + 1);
        }
        maxLen = Math.Max(maxLen, dp[i]);
    }
    return maxLen;
}
```

Explanation:
For each element, find LIS ending there by checking previous smaller elements.

---

5. Coin Change Problem (Minimum Coins to Make Amount)

```
int CoinChange(int[] coins, int amount)
{
    int[] dp = new int[amount + 1];
    Array.Fill(dp, amount + 1);
    dp[0] = 0;

    for (int i = 1; i <= amount; i++)
    {
        foreach (int coin in coins)
        {
            if (coin <= i)
                dp[i] = Math.Min(dp[i], 1 + dp[i - coin]);
        }
    }
    return dp[amount] > amount ? -1 : dp[amount];
}
```

Explanation:
Bottom-up DP: min coins needed for all amounts up to target.

---

6. Word Break Problem

```
bool WordBreak(string s, HashSet<string> wordDict)
{
    bool[] dp = new bool[s.Length + 1];
    dp[0] = true;

    for (int i = 1; i <= s.Length; i++)
    {
        for (int j = 0; j < i; j++)
        {
            if (dp[j] && wordDict.Contains(s.Substring(j, i - j)))
            {
                dp[i] = true;
                break;
            }
        }
    }
    return dp[s.Length];
}
```

Explanation:
DP to check if substring can be segmented using dictionary words.

---

7. Maximum Subarray Sum (Kadane's Algorithm)

```
int MaxSubArray(int[] nums)

{

    int maxSoFar = nums[0];

    int maxEndingHere = nums[0];


    for (int i = 1; i < nums.Length; i++)

    {

        maxEndingHere = Math.Max(nums[i], maxEndingHere + nums[i]);

        maxSoFar = Math.Max(maxSoFar, maxEndingHere);

    }

    return maxSoFar;

}
```

Explanation:
Track max subarray ending at current position; update global max.

---

8. Rod Cutting Problem

```
int RodCutting(int[] prices, int n)

{

    int[] dp = new int[n + 1];

    dp[0] = 0;
```

```
    for (int i = 1; i <= n; i++)
    {
        int maxVal = int.MinValue;
        for (int j = 0; j < i; j++)
        {
            maxVal = Math.Max(maxVal, prices[j] + dp[i - j - 1]);
        }
        dp[i] = maxVal;
    }
    return dp[n];
}
```

Explanation:
Max revenue by cutting rod into pieces of various lengths.

---

9. Matrix Chain Multiplication

```
int MatrixChainOrder(int[] dims)
{
    int n = dims.Length - 1;
    int[,] dp = new int[n, n];

    for (int l = 2; l <= n; l++)
    {
```

```
        for (int i = 0; i < n - l + 1; i++)
        {
            int j = i + l - 1;
            dp[i, j] = int.MaxValue;

            for (int k = i; k < j; k++)
            {
                int cost = dp[i, k] + dp[k + 1, j] + dims[i] *
dims[k + 1] * dims[j + 1];
                dp[i, j] = Math.Min(dp[i, j], cost);
            }
        }
    }
    return dp[0, n - 1];
}
```

Explanation:
DP calculates minimal cost to multiply chain of matrices by trying all partitions.

---

10. Partition Problem (Equal Sum Subset)

```
bool CanPartition(int[] nums)
{
    int sum = nums.Sum();
```

```
    if (sum % 2 != 0) return false;

    int target = sum / 2;
    bool[] dp = new bool[target + 1];
    dp[0] = true;

    foreach (int num in nums)
    {
        for (int j = target; j >= num; j--)
        {
            dp[j] = dp[j] || dp[j - num];
        }
    }
    return dp[target];
}
```

Explanation:
Subset sum to check if half the total sum is achievable.

Sorting and Searching

1. Quicksort

```
void QuickSort(int[] arr, int low, int high)
{
    if (low < high)
```

```
    {
        int pi = Partition(arr, low, high);
        QuickSort(arr, low, pi - 1);
        QuickSort(arr, pi + 1, high);
    }
}

int Partition(int[] arr, int low, int high)
{
    int pivot = arr[high];
    int i = low - 1;

    for (int j = low; j < high; j++)
    {
        if (arr[j] < pivot)
        {
            i++;
            (arr[i], arr[j]) = (arr[j], arr[i]);
        }
    }
    (arr[i + 1], arr[high]) = (arr[high], arr[i + 1]);
    return i + 1;
}
```

Explanation:
Choose last element as pivot, partition array so left < pivot < right, recursively sort subarrays.

---

2. Mergesort

```
void MergeSort(int[] arr, int left, int right)
{
    if (left < right)
    {
        int mid = (left + right) / 2;
        MergeSort(arr, left, mid);
        MergeSort(arr, mid + 1, right);
```

```
        Merge(arr, left, mid, right);
    }
}

void Merge(int[] arr, int left, int mid, int right)
{
    int n1 = mid - left + 1;
    int n2 = right - mid;
    int[] L = new int[n1];
    int[] R = new int[n2];

    Array.Copy(arr, left, L, 0, n1);
    Array.Copy(arr, mid + 1, R, 0, n2);

    int i = 0, j = 0, k = left;
    while (i < n1 && j < n2)
        arr[k++] = (L[i] <= R[j]) ? L[i++] : R[j++];

    while (i < n1) arr[k++] = L[i++];
    while (j < n2) arr[k++] = R[j++];
}
```

Explanation:
Divide array, sort left & right halves, then merge sorted halves.

---

3. First Occurrence of a Number in a Sorted Array

```
int FirstOccurrence(int[] arr, int target)
{
    int low = 0, high = arr.Length - 1, result = -1;

    while (low <= high)
    {
        int mid = low + (high - low) / 2;
        if (arr[mid] == target)
        {
            result = mid;
```

```
            high = mid - 1;  // search left side
        }
        else if (arr[mid] < target)
            low = mid + 1;
        else
            high = mid - 1;
    }
    return result;
}
```

Explanation:
Binary search but continue left to find first occurrence.

---

4. Find Peak Element

```
int FindPeakElement(int[] nums)
{
    int left = 0, right = nums.Length - 1;

    while (left < right)
    {
        int mid = (left + right) / 2;
        if (nums[mid] > nums[mid + 1])
            right = mid;
        else
            left = mid + 1;
    }
    return left;
}
```

Explanation:
Binary search comparing mid element with right neighbor to find peak.

---

5. Binary Search

```
int BinarySearch(int[] arr, int target)
{
    int low = 0, high = arr.Length - 1;

    while (low <= high)
    {
        int mid = low + (high - low) / 2;
        if (arr[mid] == target)
            return mid;
        else if (arr[mid] < target)
            low = mid + 1;
        else
            high = mid - 1;
    }
    return -1;
}
```

Explanation:
Standard binary search to find target's index or -1 if not found.

---

6. Find Element that Appears Once in Sorted Array (Others Appear Twice)

```
int SingleNonDuplicate(int[] nums)
{
    int low = 0, high = nums.Length - 1;

    while (low < high)
    {
        int mid = low + (high - low) / 2;
        if (mid % 2 == 1) mid--;  // ensure mid is even

        if (nums[mid] == nums[mid + 1])
            low = mid + 2;
        else
            high = mid;
```

```
    }
    return nums[low];
}
```

Explanation:
Pairs appear consecutively; use binary search on even indices to find mismatch.

---

7. Find Majority Element (Boyer-Moore Voting Algorithm)

```
int MajorityElement(int[] nums)
{
    int count = 0, candidate = 0;

    foreach (var num in nums)
    {
        if (count == 0)
            candidate = num;
        count += (num == candidate) ? 1 : -1;
    }
    return candidate;
}
```

Explanation:
Maintain a candidate and count; majority element survives this cancellation.

---

8. Find kth Smallest Element in a Sorted Matrix

```
int KthSmallest(int[][] matrix, int k)
{
    int n = matrix.Length;
    int low = matrix[0][0], high = matrix[n - 1][n - 1];

    while (low < high)
    {
```

```
        int mid = low + (high - low) / 2;
        int count = 0, j = n - 1;

        for (int i = 0; i < n; i++)
        {
            while (j >= 0 && matrix[i][j] > mid)
                j--;
            count += (j + 1);
        }

        if (count < k)
            low = mid + 1;
        else
            high = mid;
    }
    return low;
}
```

Explanation:
Binary search on values, count how many elements ≤ mid using matrix's sorted rows/cols.

---

9. Sort Array of 0s, 1s, and 2s (Dutch National Flag)

```
void SortColors(int[] nums)
{
    int low = 0, mid = 0, high = nums.Length - 1;

    while (mid <= high)
    {
        if (nums[mid] == 0)
            (nums[low++], nums[mid++]) = (nums[mid], nums[low]);
        else if (nums[mid] == 1)
            mid++;
        else
            (nums[mid], nums[high--]) = (nums[high], nums[mid]);
    }
}
```

Explanation:
Partition array into three parts in one pass using three pointers.

---

10. Find Missing Number in Sorted Array of Distinct Integers

```
int FindMissingNumber(int[] nums)
{
    int low = 0, high = nums.Length - 1;

    while (low <= high)
    {
        int mid = low + (high - low) / 2;

        if (nums[mid] == mid)
            low = mid + 1;
        else
            high = mid - 1;
    }
    return low;
}
```

Explanation:
In perfect array nums[i] == i; missing number breaks this property, use binary search to find breakpoint.

Mathematical Problems

1. Greatest Common Divisor (GCD) — Euclidean Algorithm

```
int GCD(int a, int b)
{
```

```
    while (b != 0)
    {
        int temp = b;
        b = a % b;
        a = temp;
    }
    return a;
}
```

Explanation:
Repeatedly replace (a, b) with (b, a mod b) until b is 0; then a is the GCD.

---

2. Least Common Multiple (LCM)

```
int LCM(int a, int b)
{
    return a / GCD(a, b) * b;
}
```

Explanation:
LCM × GCD = product of the two numbers.

---

3. Check if a Number is Prime

```
bool IsPrime(int n)
{
    if (n <= 1) return false;
    if (n <= 3) return true;

    if (n % 2 == 0 || n % 3 == 0) return false;

    for (int i = 5; i * i <= n; i += 6)
    {
        if (n % i == 0 || n % (i + 2) == 0)
            return false;
```

```
    }
    return true;
}
```

Explanation:
Check divisibility by 2, 3, then test possible divisors of form 6k ± 1 up to √n.

---

4. Generate All Prime Numbers up to N (Sieve of Eratosthenes)

```
List<int> GeneratePrimes(int n)
{
    bool[] isPrime = new bool[n + 1];
    for (int i = 2; i <= n; i++) isPrime[i] = true;

    for (int i = 2; i * i <= n; i++)
    {
        if (isPrime[i])
        {
            for (int j = i * i; j <= n; j += i)
                isPrime[j] = false;
        }
    }

    List<int> primes = new List<int>();
    for (int i = 2; i <= n; i++)
    {
        if (isPrime[i]) primes.Add(i);
    }
    return primes;
}
```

Explanation:
Mark multiples of each prime as non-prime starting from its square.

---

5. Sum of Digits of a Number

```
int SumOfDigits(int n)
{
    int sum = 0;
    n = Math.Abs(n);

    while (n > 0)
    {
        sum += n % 10;
        n /= 10;
    }
    return sum;
}
```

Explanation:
Extract digits using modulo 10 and add.

---

6. Count Trailing Zeroes in Factorial of a Number

```
int TrailingZeroes(int n)
{
    int count = 0;
    for (int i = 5; i <= n; i *= 5)
    {
        count += n / i;
    }
    return count;
}
```

Explanation:
Trailing zeros come from factors of 10 = 2 × 5, but 2s are plenty, count 5s.

---

7. Power of a Number (x^n) — Fast Exponentiation

```
double Power(double x, int n)
```

```
{
    if (n == 0) return 1;
    double temp = Power(x, n / 2);
    if (n % 2 == 0)
        return temp * temp;
    else
        return (n > 0) ? x * temp * temp : (temp * temp) / x;
}
```

Explanation:
Recursive fast power divides exponent by 2 to reduce complexity to O(log n).

---

8. Count Number of 1's in Binary Representation

```
int CountOnes(int n)
{
    int count = 0;
    while (n != 0)
    {
        n &= (n - 1); // drops the lowest set bit
        count++;
    }
    return count;
}
```

Explanation:
Brian Kernighan's algorithm removes one set bit per iteration.

---

9. Basic Calculator (Add, Subtract, Multiply, Divide)

```
double Calculate(double a, double b, char op)
{
    return op switch
    {
        '+' => a + b,
```

```
        '-' => a - b,
        '*' => a * b,
        '/' => b != 0 ? a / b : throw new DivideByZeroException(),
        _ => throw new ArgumentException("Invalid operator"),
    };
}
```

Explanation:
Simple switch statement performing basic arithmetic, with divide-by-zero check.

---

10. Find Largest Prime Factor of a Number

```
long LargestPrimeFactor(long n)
{
    long maxPrime = -1;

    while (n % 2 == 0)
    {
        maxPrime = 2;
        n /= 2;
    }

    for (long i = 3; i * i <= n; i += 2)
    {
        while (n % i == 0)
        {
            maxPrime = i;
            n /= i;
        }
    }

    if (n > 2) maxPrime = n;

    return maxPrime;
}
```

Explanation:
Divide out factors of 2, then test odd factors; leftover > 2 is prime.

Miscellaneous Problems

1. Count Inversions in an Array

```
int MergeSortAndCount(int[] arr, int[] temp, int left, int right)
{
    int invCount = 0;
    if (right > left)
    {
        int mid = (right + left) / 2;
        invCount += MergeSortAndCount(arr, temp, left, mid);
        invCount += MergeSortAndCount(arr, temp, mid + 1, right);
        invCount += Merge(arr, temp, left, mid + 1, right);
    }
    return invCount;
}

int Merge(int[] arr, int[] temp, int left, int mid, int right)
{
    int i = left, j = mid, k = left;
    int invCount = 0;

    while (i <= mid - 1 && j <= right)
    {
        if (arr[i] <= arr[j])
            temp[k++] = arr[i++];
        else
        {
            temp[k++] = arr[j++];
            invCount += (mid - i); // Count inversions
        }
    }
    while (i <= mid - 1)
        temp[k++] = arr[i++];
```

```
    while (j <= right)
        temp[k++] = arr[j++];

    for (int idx = left; idx <= right; idx++)
        arr[idx] = temp[idx];

    return invCount;
}
```

Explanation:
Using a modified merge sort to count pairs where arr[i] > arr[j] for i < j efficiently.

---

2. Find the Longest Palindromic Substring

```
string LongestPalindrome(string s)
{
    if (string.IsNullOrEmpty(s)) return "";

    int start = 0, maxLen = 1;

    for (int i = 0; i < s.Length; i++)
    {
        ExpandAroundCenter(s, i, i, ref start, ref maxLen);   // Odd
length palindrome
        ExpandAroundCenter(s, i, i + 1, ref start, ref maxLen); //
Even length palindrome
    }

    return s.Substring(start, maxLen);
}

void ExpandAroundCenter(string s, int left, int right, ref int
start, ref int maxLen)
{
    while (left >= 0 && right < s.Length && s[left] == s[right])
    {
        if (right - left + 1 > maxLen)
```

```
        {
            start = left;
            maxLen = right - left + 1;
        }
        left--;
        right++;
    }
}
```

Explanation:
Expand around each center to find the longest palindrome in O(n²).

---

3. Find All Subsets (Power Set)

```
List<List<int>> Subsets(int[] nums)
{
    List<List<int>> result = new List<List<int>>();
    GenerateSubsets(nums, 0, new List<int>(), result);
    return result;
}

void GenerateSubsets(int[] nums, int index, List<int> current,
List<List<int>> result)
{
    if (index == nums.Length)
    {
        result.Add(new List<int>(current));
        return;
    }

    // Exclude nums[index]
    GenerateSubsets(nums, index + 1, current, result);

    // Include nums[index]
    current.Add(nums[index]);
    GenerateSubsets(nums, index + 1, current, result);
    current.RemoveAt(current.Count - 1);
```

```
}
```

Explanation:
Backtracking approach includes/excludes each element.

---

4. Solve N-Queens Problem

```
List<List<string>> SolveNQueens(int n)
{
    List<List<string>> results = new List<List<string>>();
    int[] board = new int[n]; // board[i] = column position of queen
in row i
    Solve(0, board, results, n);
    return results;
}

void Solve(int row, int[] board, List<List<string>> results, int n)
{
    if (row == n)
    {
        results.Add(GenerateBoard(board, n));
        return;
    }

    for (int col = 0; col < n; col++)
    {
        if (IsSafe(row, col, board))
        {
            board[row] = col;
            Solve(row + 1, board, results, n);
        }
    }
}

bool IsSafe(int row, int col, int[] board)
{
    for (int i = 0; i < row; i++)
```

```
    {
        if (board[i] == col || Math.Abs(board[i] - col) ==
Math.Abs(i - row))
            return false;
    }
    return true;
}

List<string> GenerateBoard(int[] board, int n)
{
    List<string> res = new List<string>();
    for (int i = 0; i < n; i++)
    {
        char[] row = new char[n];
        for (int j = 0; j < n; j++)
            row[j] = '.';
        row[board[i]] = 'Q';
        res.Add(new string(row));
    }
    return res;
}
```

Explanation:
Backtracking places queens row by row while checking columns and diagonals.

---

5. Find Number of Islands in 2D Matrix

```
int NumIslands(char[][] grid)
{
    if (grid == null || grid.Length == 0) return 0;
    int count = 0;

    for (int i = 0; i < grid.Length; i++)
    {
        for (int j = 0; j < grid[0].Length; j++)
        {
            if (grid[i][j] == '1')
```

```
            {
                DFS(grid, i, j);
                count++;
            }
        }
    }
    return count;
}

void DFS(char[][] grid, int i, int j)
{
    if (i < 0 || j < 0 || i >= grid.Length || j >= grid[0].Length ||
grid[i][j] == '0')
        return;

    grid[i][j] = '0'; // Mark visited
    DFS(grid, i + 1, j);
    DFS(grid, i - 1, j);
    DFS(grid, i, j + 1);
    DFS(grid, i, j - 1);
}
```

Explanation:
Use DFS to mark all connected land cells, count islands by visiting unvisited lands.

---

6. Find Longest Consecutive Sequence in an Unsorted Array

```
int LongestConsecutive(int[] nums)
{
    HashSet<int> set = new HashSet<int>(nums);
    int longest = 0;

    foreach (int num in set)
    {
        if (!set.Contains(num - 1))
```

```
        {
            int currentNum = num;
            int length = 1;

            while (set.Contains(currentNum + 1))
            {
                currentNum++;
                length++;
            }
            longest = Math.Max(longest, length);
        }
    }
    return longest;
}
```

Explanation:
Check only starts of sequences, count consecutive numbers using HashSet for O(n).

---

7. Find Majority Element (More than n/2 times)

```
int MajorityElement(int[] nums)
{
    int count = 0, candidate = 0;

    foreach (var num in nums)
    {
        if (count == 0)
            candidate = num;
        count += (num == candidate) ? 1 : -1;
    }
    return candidate;
}
```

Explanation:
Boyer-Moore Voting Algorithm tracks majority element by counting net votes.

---

8. Rotate Matrix by 90 Degrees (Clockwise)

```
void RotateMatrix(int[][] matrix)
{
    int n = matrix.Length;

    // Transpose
    for (int i = 0; i < n; i++)
        for (int j = i; j < n; j++)
            (matrix[i][j], matrix[j][i]) = (matrix[j][i],
matrix[i][j]);

    // Reverse each row
    for (int i = 0; i < n; i++)
    {
        int left = 0, right = n - 1;
        while (left < right)
        {
            (matrix[i][left], matrix[i][right]) = (matrix[i][right],
matrix[i][left]);
            left++;
            right--;
        }
    }
}
```

Explanation:
Transpose matrix and then reverse each row to rotate clockwise by 90°.

---

9. Solve Sudoku (Backtracking)

```
bool SolveSudoku(char[][] board)
{
    for (int i = 0; i < 9; i++)
    {
        for (int j = 0; j < 9; j++)
        {
```

```
            if (board[i][j] == '.')
            {
                for (char c = '1'; c <= '9'; c++)
                {
                    if (IsValid(board, i, j, c))
                    {
                        board[i][j] = c;
                        if (SolveSudoku(board))
                            return true;
                        else
                            board[i][j] = '.';
                    }
                }
                return false;
            }
        }
    }
    return true;
}

bool IsValid(char[][] board, int row, int col, char c)
{
    for (int i = 0; i < 9; i++)
    {
        if (board[row][i] == c) return false;
        if (board[i][col] == c) return false;
        if (board[3 * (row / 3) + i / 3][3 * (col / 3) + i % 3] ==
c) return false;
    }
    return true;
}
```


Explanation:
Backtracking tries digits 1-9 in empty cells, validating constraints.

---

10. Generate Parentheses (Well-formed Combinations)

```
List<string> GenerateParenthesis(int n)
{
    List<string> result = new List<string>();
    Generate("", 0, 0, n, result);
    return result;
}

void Generate(string current, int open, int close, int max,
List<string> result)
{
    if (current.Length == max * 2)
    {
        result.Add(current);
        return;
    }
    if (open < max)
        Generate(current + "(", open + 1, close, max, result);
    if (close < open)
        Generate(current + ")", open, close + 1, max, result);
}
```

Explanation:
Use backtracking to add '(' and ')' only when valid.

1. Implement `strStr()` — Find first occurrence of a substring (needle) in a string (haystack)

```
int StrStr(string haystack, string needle)
{
    if (needle == "") return 0;
    int n = haystack.Length, m = needle.Length;

    for (int i = 0; i <= n - m; i++)
    {
        int j;
```

```
        for (j = 0; j < m; j++)
        {
            if (haystack[i + j] != needle[j])
                break;
        }
        if (j == m) return i; // found
    }
    return -1;
}
```

Explanation:
Simple sliding window check each position; returns starting index or -1.

---

2. Find the Longest Palindromic Substring

(Repeated here for convenience)

```
string LongestPalindrome(string s)
{
    if (string.IsNullOrEmpty(s)) return "";

    int start = 0, maxLen = 1;

    for (int i = 0; i < s.Length; i++)
    {
        ExpandAroundCenter(s, i, i, ref start, ref maxLen);
        ExpandAroundCenter(s, i, i + 1, ref start, ref maxLen);
    }

    return s.Substring(start, maxLen);
}

void ExpandAroundCenter(string s, int left, int right, ref int
start, ref int maxLen)
{
    while (left >= 0 && right < s.Length && s[left] == s[right])
    {
```

```
        if (right - left + 1 > maxLen)
        {
            start = left;
            maxLen = right - left + 1;
        }
        left--;
        right++;
    }
}
```

---

3. Regular Expression Matching (. and *)

```
bool IsMatch(string s, string p)
{
    return IsMatchHelper(s, p, 0, 0);
}

bool IsMatchHelper(string s, string p, int i, int j)
{
    if (j == p.Length) return i == s.Length;

    bool firstMatch = (i < s.Length) && (p[j] == s[i] || p[j] ==
'.');

    if (j + 1 < p.Length && p[j + 1] == '*')
    {
        // Two cases:
        // 1) Use zero occurrence of p[j] (skip)
        // 2) If firstMatch, consume one char in s and keep pattern
at j
        return IsMatchHelper(s, p, i, j + 2) ||
               (firstMatch && IsMatchHelper(s, p, i + 1, j));
    }
    else
    {
        return firstMatch && IsMatchHelper(s, p, i + 1, j + 1);
    }
```

```
}
```

Explanation:
Recursively matches strings supporting '.' (any char) and '*' (zero or more of preceding).

---

4. Count Number of Occurrences of Each Character in a String

```
Dictionary<char, int> CountCharacters(string s)
{
    var dict = new Dictionary<char, int>();
    foreach (char c in s)
    {
        if (dict.ContainsKey(c))
            dict[c]++;
        else
            dict[c] = 1;
    }
    return dict;
}
```

Explanation:
Simple frequency map using Dictionary.

---

5. Check if a String is Rotation of Another String

```
bool IsRotation(string s1, string s2)
{
    if (s1.Length != s2.Length) return false;
    string doubled = s1 + s1;
    return doubled.Contains(s2);
}
```

Explanation:
If s2 is rotation of s1, it must be substring of s1+s1.

---

6. Find the Smallest Window in String `s` that Contains All Characters of String `t`

```
string MinWindow(string s, string t)
{
    if (string.IsNullOrEmpty(s) || string.IsNullOrEmpty(t)) return
"";

    Dictionary<char, int> dictT = new Dictionary<char, int>();
    foreach (char c in t)
        dictT[c] = dictT.ContainsKey(c) ? dictT[c] + 1 : 1;

    int required = dictT.Count;
    int formed = 0;

    Dictionary<char, int> windowCounts = new Dictionary<char,
int>();
    int left = 0, right = 0;
    int minLen = int.MaxValue, minLeft = 0;

    while (right < s.Length)
    {
        char c = s[right];
        windowCounts[c] = windowCounts.ContainsKey(c) ?
windowCounts[c] + 1 : 1;

        if (dictT.ContainsKey(c) && windowCounts[c] == dictT[c])
            formed++;

        while (left <= right && formed == required)
        {
            if (right - left + 1 < minLen)
            {
                minLen = right - left + 1;
```

```
                minLeft = left;
            }

            char leftChar = s[left];
            windowCounts[leftChar]--;
            if (dictT.ContainsKey(leftChar) &&
windowCounts[leftChar] < dictT[leftChar])
                formed--;

            left++;
        }
        right++;
    }

    return minLen == int.MaxValue ? "" : s.Substring(minLeft,
minLen);
}
```

Explanation:
Sliding window with two pointers keeps track of counts of chars matching the target.

1. Detect a cycle in a linked list and return the node where the cycle begins

```
public class ListNode {
    public int val;
    public ListNode next;
    public ListNode(int x) { val = x; next = null; }
}

ListNode DetectCycle(ListNode head) {
    if (head == null) return null;

    ListNode slow = head, fast = head;
```

```
    // Detect cycle using Floyd's Tortoise and Hare
    while (fast != null && fast.next != null) {
        slow = slow.next;
        fast = fast.next.next;
        if (slow == fast) { // cycle detected
            ListNode ptr = head;
            while (ptr != slow) {
                ptr = ptr.next;
                slow = slow.next;
            }
            return ptr; // start node of cycle
        }
    }
    return null; // no cycle
}
```

Explanation:
First detect cycle meeting point with two pointers. Then find start node by moving one pointer from head and one from meeting point until they meet.

---

2. Reverse a portion of a linked list (from position m to n)

```
ListNode ReverseBetween(ListNode head, int m, int n) {
    if (head == null || m == n) return head;

    ListNode dummy = new ListNode(0);
    dummy.next = head;
    ListNode prev = dummy;

    // Move prev to one before m-th node
    for (int i = 1; i < m; i++) prev = prev.next;

    ListNode start = prev.next;
    ListNode then = start.next;

    // Reverse the sublist
    for (int i = 0; i < n - m; i++) {
```

```
        start.next = then.next;
        then.next = prev.next;
        prev.next = then;
        then = start.next;
    }
    return dummy.next;
}
```

Explanation:
Use a dummy node to simplify edge cases. Reverse nodes between m and n by changing pointers.

---

3. Merge k sorted linked lists into one sorted linked list

```
ListNode MergeKLists(ListNode[] lists) {
    if (lists == null || lists.Length == 0) return null;

    PriorityQueue<ListNode, int> pq = new PriorityQueue<ListNode,
int>();

    foreach (var list in lists)
        if (list != null)
            pq.Enqueue(list, list.val);

    ListNode dummy = new ListNode(0);
    ListNode current = dummy;

    while (pq.Count > 0) {
        var node = pq.Dequeue();
        current.next = node;
        current = current.next;
        if (node.next != null)
            pq.Enqueue(node.next, node.next.val);
    }
    return dummy.next;
}
```

Explanation:
Use a min-heap (priority queue) to always pick the smallest head node among k lists.

---

4. Find the intersection node of two linked lists (if any)

```
ListNode GetIntersectionNode(ListNode headA, ListNode headB) {
    if (headA == null || headB == null) return null;

    ListNode a = headA, b = headB;

    while (a != b) {
        a = (a == null) ? headB : a.next;
        b = (b == null) ? headA : b.next;
    }
    return a; // either intersection or null
}
```

Explanation:
Two pointers traverse both lists; if no intersection, both will reach null simultaneously.

---

5. Add two numbers represented by linked lists (each node contains a digit)

```
ListNode AddTwoNumbers(ListNode l1, ListNode l2) {
    ListNode dummy = new ListNode(0);
    ListNode curr = dummy;
    int carry = 0;

    while (l1 != null || l2 != null || carry != 0) {
        int x = (l1 != null) ? l1.val : 0;
        int y = (l2 != null) ? l2.val : 0;
        int sum = x + y + carry;
        carry = sum / 10;
        curr.next = new ListNode(sum % 10);
        curr = curr.next;
        if (l1 != null) l1 = l1.next;
        if (l2 != null) l2 = l2.next;
```

```
    }
    return dummy.next;
}
```

Explanation:
Add digit by digit with carry, creating new nodes for the result.

---

6. Flatten a linked list with next and child pointers

```
public class MultiLevelNode {
    public int val;
    public MultiLevelNode next;
    public MultiLevelNode child;
    public MultiLevelNode(int x) { val = x; next = null; child =
null; }
}

MultiLevelNode Flatten(MultiLevelNode head) {
    if (head == null) return null;

    MultiLevelNode dummy = new MultiLevelNode(0);
    MultiLevelNode prev = dummy;
    Stack<MultiLevelNode> stack = new Stack<MultiLevelNode>();
    stack.Push(head);

    while (stack.Count > 0) {
        var curr = stack.Pop();
        prev.next = curr;
        curr.child = null; // remove child pointer after flattening
        prev = curr;

        if (curr.next != null) stack.Push(curr.next);
        if (curr.child != null) stack.Push(curr.child);
    }

    return dummy.next;
}
```

Explanation:
Use a stack to perform DFS; attach nodes and remove child pointers.

1. Implement a queue using stacks (efficient enqueue and dequeue)

```
public class MyQueue {
    private Stack<int> stackIn = new Stack<int>();
    private Stack<int> stackOut = new Stack<int>();

    // Enqueue: push into stackIn (O(1))
    public void Enqueue(int x) {
        stackIn.Push(x);
    }

    // Dequeue: if stackOut empty, pour all from stackIn to
stackOut, then pop (amortized O(1))
    public int Dequeue() {
        if (stackOut.Count == 0) {
            while (stackIn.Count > 0) {
                stackOut.Push(stackIn.Pop());
            }
        }
        return stackOut.Pop();
    }

    public int Peek() {
        if (stackOut.Count == 0) {
            while (stackIn.Count > 0) {
                stackOut.Push(stackIn.Pop());
            }
        }
        return stackOut.Peek();
    }

    public bool IsEmpty() {
        return stackIn.Count == 0 && stackOut.Count == 0;
    }
}
```

Explanation:
Two stacks are used: `stackIn` for enqueue, `stackOut` for dequeue. Elements are transferred only when needed, making both operations amortized O(1).

---

2. Evaluate an infix expression (with parentheses)

```
public int EvaluateInfix(string expression) {
    Stack<int> operands = new Stack<int>();
    Stack<char> operators = new Stack<char>();

    int i = 0;
    while (i < expression.Length) {
        if (char.IsWhiteSpace(expression[i])) {
            i++;
            continue;
        }

        if (char.IsDigit(expression[i])) {
            int val = 0;
            while (i < expression.Length &&
char.IsDigit(expression[i])) {
                val = val * 10 + (expression[i] - '0');
                i++;
            }
            operands.Push(val);
            continue;
        }

        if (expression[i] == '(') {
            operators.Push(expression[i]);
        }
        else if (expression[i] == ')') {
            while (operators.Peek() != '(') {
                ApplyOp(operands, operators);
            }
            operators.Pop(); // remove '('
```

```
        }
        else if (IsOperator(expression[i])) {
            while (operators.Count > 0 &&
Precedence(operators.Peek()) >= Precedence(expression[i])) {
                ApplyOp(operands, operators);
            }
            operators.Push(expression[i]);
        }
        i++;
    }

    while (operators.Count > 0) {
        ApplyOp(operands, operators);
    }

    return operands.Pop();
}

bool IsOperator(char c) {
    return c == '+' || c == '-' || c == '*' || c == '/';
}

int Precedence(char op) {
    if (op == '+' || op == '-') return 1;
    if (op == '*' || op == '/') return 2;
    return 0;
}

void ApplyOp(Stack<int> operands, Stack<char> operators) {
    int b = operands.Pop();
    int a = operands.Pop();
    char op = operators.Pop();

    int result = 0;
    switch (op) {
        case '+': result = a + b; break;
        case '-': result = a - b; break;
        case '*': result = a * b; break;
```

```
            case '/': result = a / b; break;
        }
        operands.Push(result);
    }
```

Explanation:
Standard two-stack algorithm for evaluating infix expressions considering operator precedence and parentheses.

---

3. Find the next greater element for every element in an array

```
int[] NextGreaterElements(int[] nums) {
    int n = nums.Length;
    int[] result = new int[n];
    Stack<int> stack = new Stack<int>();

    for (int i = n - 1; i >= 0; i--) {
        while (stack.Count > 0 && stack.Peek() <= nums[i]) {
            stack.Pop();
        }
        result[i] = stack.Count == 0 ? -1 : stack.Peek();
        stack.Push(nums[i]);
    }

    return result;
}
```

Explanation:
Traverse from right to left, use stack to keep track of next greater elements in O(n).

---

4. Design a stack to support push, pop, and retrieving the minimum element in constant time

```
public class MinStack {
    private Stack<int> stack = new Stack<int>();
    private Stack<int> minStack = new Stack<int>();
```

```
    public void Push(int x) {
        stack.Push(x);
        if (minStack.Count == 0 || x <= minStack.Peek())
            minStack.Push(x);
    }

    public void Pop() {
        if (stack.Peek() == minStack.Peek())
            minStack.Pop();
        stack.Pop();
    }

    public int Top() {
        return stack.Peek();
    }

    public int GetMin() {
        return minStack.Peek();
    }
}
```

Explanation:
Use two stacks: one normal stack, one for minimum values. When pushing a smaller or equal value, push it on minStack; when popping, pop minStack if needed.

---

5. Implement sliding window maximum (using a deque)

```
public int[] MaxSlidingWindow(int[] nums, int k) {
    if (nums == null || k <= 0) return new int[0];
    int n = nums.Length;
    int[] result = new int[n - k + 1];
    LinkedList<int> deque = new LinkedList<int>(); // store indices

    for (int i = 0; i < n; i++) {
        // Remove indices out of window
        if (deque.Count > 0 && deque.First.Value <= i - k)
```

```
            deque.RemoveFirst();

        // Remove smaller values from the back
        while (deque.Count > 0 && nums[deque.Last.Value] < nums[i])
            deque.RemoveLast();

        deque.AddLast(i);

        if (i >= k - 1)
            result[i - k + 1] = nums[deque.First.Value];
    }

    return result;
}
```

Explanation:
Use a deque to keep indexes of useful elements in current window, ensuring the front is always max.

1. Convert a binary tree to a doubly linked list (in-order)

```
public class TreeNode {
    public int val;
    public TreeNode left, right;
    public TreeNode(int x) { val = x; }
}

TreeNode prev = null;
TreeNode head = null;

TreeNode ConvertToDLL(TreeNode root) {
    if (root == null) return null;

    ConvertToDLL(root.left);

    if (prev == null) {
        head = root; // first node becomes head
    } else {
        root.left = prev;
```

```
        prev.right = root;
    }
    prev = root;

    ConvertToDLL(root.right);

    return head;
}
```

Explanation:
Inorder traversal connects nodes as doubly linked list by linking current with previous node.

---

2. Find the vertical sum of a binary tree

```
void VerticalSum(TreeNode root, int hd, Dictionary<int, int> map) {
    if (root == null) return;
    VerticalSum(root.left, hd - 1, map);

    if (map.ContainsKey(hd))
        map[hd] += root.val;
    else
        map[hd] = root.val;

    VerticalSum(root.right, hd + 1, map);
}

Dictionary<int, int> GetVerticalSum(TreeNode root) {
    var map = new Dictionary<int, int>();
    VerticalSum(root, 0, map);
    return map;
}
```

Explanation:
Use horizontal distance (hd) from root; sum values of nodes at each hd.

---

3. Flatten a binary tree into a linked list (preorder traversal)

```
TreeNode prev = null;
void Flatten(TreeNode root) {
    if (root == null) return;

    Flatten(root.right);
    Flatten(root.left);

    root.right = prev;
    root.left = null;
    prev = root;
}
```

Explanation:
Postorder traversal (right-left-root) to flatten tree in place.

---

4. Check if two trees are identical

```
bool IsIdentical(TreeNode p, TreeNode q) {
    if (p == null && q == null) return true;
    if (p == null || q == null) return false;
    if (p.val != q.val) return false;
    return IsIdentical(p.left, q.left) && IsIdentical(p.right,
q.right);
}
```

Explanation:
Recursive check values and structure for equality.

---

5. Find the distance between two nodes in a binary tree

```
TreeNode LCA(TreeNode root, int n1, int n2) {
    if (root == null) return null;
    if (root.val == n1 || root.val == n2) return root;

    TreeNode left = LCA(root.left, n1, n2);
```

```
    TreeNode right = LCA(root.right, n1, n2);

    if (left != null && right != null) return root;
    return left ?? right;
}

int FindLevel(TreeNode root, int val, int level) {
    if (root == null) return -1;
    if (root.val == val) return level;

    int left = FindLevel(root.left, val, level + 1);
    if (left != -1) return left;
    return FindLevel(root.right, val, level + 1);
}

int DistanceBetweenNodes(TreeNode root, int n1, int n2) {
    TreeNode lca = LCA(root, n1, n2);
    int d1 = FindLevel(lca, n1, 0);
    int d2 = FindLevel(lca, n2, 0);
    return d1 + d2;
}
```

Explanation:
Find Lowest Common Ancestor (LCA) then sum distances from LCA to each node.

---

6. Level-order traversal but return values in reverse order

```
List<List<int>> LevelOrderBottom(TreeNode root) {
    var res = new List<List<int>>();
    if (root == null) return res;

    Queue<TreeNode> queue = new Queue<TreeNode>();
    queue.Enqueue(root);

    while (queue.Count > 0) {
        int size = queue.Count;
        var level = new List<int>();
```

```
        for (int i = 0; i < size; i++) {
            TreeNode node = queue.Dequeue();
            level.Add(node.val);
            if (node.left != null) queue.Enqueue(node.left);
            if (node.right != null) queue.Enqueue(node.right);
        }
        res.Insert(0, level); // prepend to get reverse order
    }
    return res;
}
```

Explanation:
Perform normal BFS, insert each level at front of result list for reversed order.

---

7. Print all ancestors of a given node in a binary tree

```
bool PrintAncestors(TreeNode root, int target) {
    if (root == null) return false;
    if (root.val == target) return true;

    if (PrintAncestors(root.left, target) ||
PrintAncestors(root.right, target)) {
        Console.Write(root.val + " ");
        return true;
    }
    return false;
}
```

Explanation:
Recursive check if target is in subtree; print current node on path back if yes.

---

8. Check if a binary tree is a valid BST (with recursion)

```
bool IsValidBST(TreeNode root) {
    return Validate(root, null, null);
}
```

```
bool Validate(TreeNode node, int? min, int? max) {
    if (node == null) return true;
    if ((min != null && node.val <= min) || (max != null && node.val
>= max)) return false;
    return Validate(node.left, min, node.val) &&
Validate(node.right, node.val, max);
}
```

Explanation:
Pass down min and max bounds for subtree values; node must be in (min, max) range.

---

9. Find the maximum width of a binary tree

```
int WidthOfBinaryTree(TreeNode root) {
    if (root == null) return 0;

    int maxWidth = 0;
    Queue<(TreeNode node, int idx)> queue = new Queue<(TreeNode,
int)>();
    queue.Enqueue((root, 0));

    while (queue.Count > 0) {
        int size = queue.Count;
        int start = queue.Peek().idx;
        int end = start;

        for (int i = 0; i < size; i++) {
            var (node, idx) = queue.Dequeue();
            end = idx;

            if (node.left != null)
                queue.Enqueue((node.left, 2 * idx + 1));
            if (node.right != null)
                queue.Enqueue((node.right, 2 * idx + 2));
        }
        maxWidth = Math.Max(maxWidth, end - start + 1);
```

```
    }
    return maxWidth;
}
```

Explanation:
Assign index to each node as if in a complete tree; width is max difference of indices per level.

1. BFS on a graph (adjacency list)

```
void BFS(Dictionary<int, List<int>> graph, int start) {
    var visited = new HashSet<int>();
    var queue = new Queue<int>();

    queue.Enqueue(start);
    visited.Add(start);

    while (queue.Count > 0) {
        int node = queue.Dequeue();
        Console.WriteLine(node);

        foreach (var neighbor in graph[node]) {
            if (!visited.Contains(neighbor)) {
                visited.Add(neighbor);
                queue.Enqueue(neighbor);
            }
        }
    }
}
```

Explanation:
Classic BFS using a queue and visited set.

---

2. Number of connected components in undirected graph

```
int CountConnectedComponents(Dictionary<int, List<int>> graph) {
    var visited = new HashSet<int>();
    int count = 0;
```

```
    foreach (var node in graph.Keys) {
        if (!visited.Contains(node)) {
            DFS(node, graph, visited);
            count++;
        }
    }
    return count;
}

void DFS(int node, Dictionary<int, List<int>> graph, HashSet<int>
visited) {
    visited.Add(node);
    foreach (var neighbor in graph[node]) {
        if (!visited.Contains(neighbor)) {
            DFS(neighbor, graph, visited);
        }
    }
}
```

Explanation:
Run DFS on unvisited nodes, count how many times DFS starts.

---

3. Bellman-Ford Algorithm (shortest path from source)

```
public class Edge {
    public int Source, Dest, Weight;
    public Edge(int s, int d, int w) {
        Source = s; Dest = d; Weight = w;
    }
}

int[] BellmanFord(int vertices, List<Edge> edges, int source) {
    int[] dist = new int[vertices];
    for (int i = 0; i < vertices; i++) dist[i] = int.MaxValue;
    dist[source] = 0;
```

```
    for (int i = 1; i < vertices; i++) {
        foreach (var edge in edges) {
            if (dist[edge.Source] != int.MaxValue &&
dist[edge.Source] + edge.Weight < dist[edge.Dest]) {
                dist[edge.Dest] = dist[edge.Source] + edge.Weight;
            }
        }
    }

    // Detect negative weight cycle (optional)
    foreach (var edge in edges) {
        if (dist[edge.Source] != int.MaxValue && dist[edge.Source] +
edge.Weight < dist[edge.Dest]) {
            throw new Exception("Graph contains negative weight
cycle");
        }
    }

    return dist;
}
```

Explanation:
Relax edges V-1 times, then check for negative weight cycles.

---

4. Find all strongly connected components (Tarjan's Algorithm)

```
List<List<int>> TarjanSCC(Dictionary<int, List<int>> graph) {
    int time = 0;
    var stack = new Stack<int>();
    var onStack = new HashSet<int>();
    var low = new Dictionary<int, int>();
    var disc = new Dictionary<int, int>();
    var visited = new HashSet<int>();
    var sccList = new List<List<int>>();

    void DFS(int u) {
        disc[u] = time;
```

```
            low[u] = time;
            time++;
            stack.Push(u);
            onStack.Add(u);
            visited.Add(u);

            foreach (var v in graph[u]) {
                if (!disc.ContainsKey(v)) {
                    DFS(v);
                    low[u] = Math.Min(low[u], low[v]);
                } else if (onStack.Contains(v)) {
                    low[u] = Math.Min(low[u], disc[v]);
                }
            }

            if (low[u] == disc[u]) {
                var scc = new List<int>();
                int w;
                do {
                    w = stack.Pop();
                    onStack.Remove(w);
                    scc.Add(w);
                } while (w != u);
                sccList.Add(scc);
            }
        }

        foreach (var node in graph.Keys) {
            if (!disc.ContainsKey(node))
                DFS(node);
        }
        return sccList;
    }
```

Explanation:
Tarjan's algorithm finds SCCs using low-link values and DFS stack.

---

5. Dijkstra's Algorithm for shortest path

```
int[] Dijkstra(Dictionary<int, List<(int neighbor, int weight)>>
graph, int source, int vertices) {
    int[] dist = new int[vertices];
    for (int i = 0; i < vertices; i++) dist[i] = int.MaxValue;
    dist[source] = 0;

    var pq = new SortedSet<(int dist, int node)>();
    pq.Add((0, source));

    while (pq.Count > 0) {
        var current = pq.Min;
        pq.Remove(current);
        int u = current.node;

        foreach (var (v, w) in graph[u]) {
            if (dist[u] + w < dist[v]) {
                if (dist[v] != int.MaxValue)
                    pq.Remove((dist[v], v));
                dist[v] = dist[u] + w;
                pq.Add((dist[v], v));
            }
        }
    }

    return dist;
}
```

Explanation:
Uses a priority queue to pick node with min dist; relax edges.

---

6. Print nodes at a specific level (BFS)

```
void PrintNodesAtLevel(Dictionary<int, List<int>> graph, int start,
int targetLevel) {
    var visited = new HashSet<int>();
    var queue = new Queue<(int node, int level)>();
```

```
    queue.Enqueue((start, 0));
    visited.Add(start);

    while (queue.Count > 0) {
        var (node, level) = queue.Dequeue();
        if (level == targetLevel) {
            Console.WriteLine(node);
        }
        if (level > targetLevel) break;

        foreach (var neighbor in graph[node]) {
            if (!visited.Contains(neighbor)) {
                visited.Add(neighbor);
                queue.Enqueue((neighbor, level + 1));
            }
        }
    }
}
```

Explanation:
BFS traversal with level tracking; print nodes at the requested level.

1. Unique Paths in a Grid

```
int UniquePaths(int m, int n) {
    int[,] dp = new int[m, n];
    for (int i = 0; i < m; i++) dp[i, 0] = 1;
    for (int j = 0; j < n; j++) dp[0, j] = 1;

    for (int i = 1; i < m; i++) {
        for (int j = 1; j < n; j++) {
            dp[i, j] = dp[i - 1, j] + dp[i, j - 1];
        }
    }
    return dp[m - 1, n - 1];
}
```

2. Climbing Stairs (1 or 2 steps)

```
int ClimbStairs(int n) {
    if (n <= 2) return n;
    int a = 1, b = 2;
    for (int i = 3; i <= n; i++) {
        int c = a + b;
        a = b;
        b = c;
    }
    return b;
}
```

---

3. House Robber

```
int HouseRobber(int[] nums) {
    if (nums.Length == 0) return 0;
    if (nums.Length == 1) return nums[0];

    int prev1 = 0, prev2 = 0;
    foreach (var num in nums) {
        int temp = prev1;
        prev1 = Math.Max(prev2 + num, prev1);
        prev2 = temp;
    }
    return prev1;
}
```

---

4. Longest Palindromic Subsequence

```
int LongestPalindromeSubseq(string s) {
    int n = s.Length;
    int[,] dp = new int[n, n];
    for (int i = n - 1; i >= 0; i--) {
        dp[i, i] = 1;
        for (int j = i + 1; j < n; j++) {
            if (s[i] == s[j])
```

```
                dp[i, j] = dp[i + 1, j - 1] + 2;
            else
                dp[i, j] = Math.Max(dp[i + 1, j], dp[i, j - 1]);
        }
    }
    return dp[0, n - 1];
}
```

---

5. Minimum Path Sum in a grid

```
int MinPathSum(int[][] grid) {
    int m = grid.Length, n = grid[0].Length;
    for (int i = 1; i < m; i++) grid[i][0] += grid[i - 1][0];
    for (int j = 1; j < n; j++) grid[0][j] += grid[0][j - 1];

    for (int i = 1; i < m; i++) {
        for (int j = 1; j < n; j++) {
            grid[i][j] += Math.Min(grid[i - 1][j], grid[i][j - 1]);
        }
    }
    return grid[m - 1][n - 1];
}
```

---

6. Edit Distance (Levenshtein Distance)

```
int EditDistance(string word1, string word2) {
    int m = word1.Length, n = word2.Length;
    int[,] dp = new int[m + 1, n + 1];

    for (int i = 0; i <= m; i++) dp[i, 0] = i;
    for (int j = 0; j <= n; j++) dp[0, j] = j;

    for (int i = 1; i <= m; i++) {
        for (int j = 1; j <= n; j++) {
            if (word1[i - 1] == word2[j - 1])
                dp[i, j] = dp[i - 1, j - 1];
            else
```

```
                dp[i, j] = 1 + Math.Min(dp[i - 1, j - 1],
Math.Min(dp[i - 1, j], dp[i, j - 1]));
            }
        }
        return dp[m, n];
}
```

---

7. Palindrome Partitioning (minimum cuts)

```
int MinCutPalindromePartition(string s) {
    int n = s.Length;
    bool[,] dp = new bool[n, n];
    int[] cuts = new int[n];

    for (int i = 0; i < n; i++) {
        int minCuts = i;
        for (int j = 0; j <= i; j++) {
            if (s[j] == s[i] && (i - j < 2 || dp[j + 1, i - 1])) {
                dp[j, i] = true;
                minCuts = j == 0 ? 0 : Math.Min(minCuts, cuts[j - 1]
+ 1);
            }
        }
        cuts[i] = minCuts;
    }
    return cuts[n - 1];
}
```

---

8. Maximum Sum Increasing Subsequence

```
int MaxSumIncreasingSubsequence(int[] nums) {
    int n = nums.Length;
    int[] dp = new int[n];
    Array.Copy(nums, dp, n);

    for (int i = 1; i < n; i++) {
        for (int j = 0; j < i; j++) {
```

```
            if (nums[i] > nums[j])
                dp[i] = Math.Max(dp[i], dp[j] + nums[i]);
        }
    }
    return dp.Max();
}
```

---

9. Word Break II (all possible sentences)

```
List<string> WordBreak(string s, IList<string> wordDict) {
    var wordSet = new HashSet<string>(wordDict);
    var memo = new Dictionary<int, List<string>>();
    return DFS(0);

    List<string> DFS(int start) {
        if (memo.ContainsKey(start)) return memo[start];

        var res = new List<string>();
        if (start == s.Length) {
            res.Add("");
            return res;
        }

        for (int end = start + 1; end <= s.Length; end++) {
            string word = s.Substring(start, end - start);
            if (wordSet.Contains(word)) {
                foreach (var sub in DFS(end)) {
                    string space = sub.Length == 0 ? "" : " ";
                    res.Add(word + space + sub);
                }
            }
        }
        memo[start] = res;
        return res;
    }
}
```

1. Find the kth largest element in an unsorted array using Quickselect

```
public int FindKthLargest(int[] nums, int k) {
    return QuickSelect(nums, 0, nums.Length - 1, nums.Length - k);
}

private int QuickSelect(int[] nums, int left, int right, int
kSmallest) {
    if (left == right) return nums[left];

    int pivotIndex = Partition(nums, left, right);

    if (kSmallest == pivotIndex)
        return nums[kSmallest];
    else if (kSmallest < pivotIndex)
        return QuickSelect(nums, left, pivotIndex - 1, kSmallest);
    else
        return QuickSelect(nums, pivotIndex + 1, right, kSmallest);
}

private int Partition(int[] nums, int left, int right) {
    int pivot = nums[right];
    int i = left;

    for (int j = left; j < right; j++) {
        if (nums[j] <= pivot) {
            Swap(nums, i, j);
            i++;
        }
    }
    Swap(nums, i, right);
    return i;
}

private void Swap(int[] nums, int i, int j) {
    int temp = nums[i];
    nums[i] = nums[j];
    nums[j] = temp;
}
```

Explanation:
Quickselect partitions the array like Quicksort and recursively searches for the kth smallest element. Here, `nums.Length - k` gives the kth largest.

---

2. Sort a nearly sorted array (each element at most k positions away)

```
public int[] SortNearlySorted(int[] nums, int k) {
    var result = new List<int>();
    var minHeap = new SortedSet<(int val, int index)>();

    for (int i = 0; i < nums.Length; i++) {
        minHeap.Add((nums[i], i));
        if (minHeap.Count > k) {
            var min = minHeap.Min;
            minHeap.Remove(min);
            result.Add(min.val);
        }
    }
    while (minHeap.Count > 0) {
        var min = minHeap.Min;
        minHeap.Remove(min);
        result.Add(min.val);
    }
    return result.ToArray();
}
```

Explanation:
Use a min-heap of size k+1 to always extract the smallest element in the current window.

---

3. Find the median of a data stream

```
public class MedianFinder {
    private PriorityQueue<int, int> maxHeap; // lower half (max
heap)
```

```
    private PriorityQueue<int, int> minHeap; // upper half (min
heap)

    public MedianFinder() {
        maxHeap = new PriorityQueue<int,
int>(Comparer<int>.Create((a, b) => b.CompareTo(a)));
        minHeap = new PriorityQueue<int, int>();
    }

    public void AddNum(int num) {
        maxHeap.Enqueue(num, num);
        minHeap.Enqueue(maxHeap.Dequeue(), maxHeap.Peek());

        if (maxHeap.Count < minHeap.Count)
            maxHeap.Enqueue(minHeap.Dequeue(), minHeap.Peek());
    }

    public double FindMedian() {
        if (maxHeap.Count > minHeap.Count)
            return maxHeap.Peek();
        return (maxHeap.Peek() + minHeap.Peek()) / 2.0;
    }
}
```

Explanation:
Maintain two heaps: maxHeap for lower half, minHeap for upper half. Balance their sizes.

---

4. Number of ways to partition an array into subsets with equal sum

```
public int CountPartitions(int[] nums) {
    int sum = 0;
    foreach (int num in nums) sum += num;
    if (sum % 2 != 0) return 0;

    int target = sum / 2;
    int[] dp = new int[target + 1];
    dp[0] = 1;
```

```
    foreach (int num in nums) {
        for (int j = target; j >= num; j--) {
            dp[j] += dp[j - num];
        }
    }
    return dp[target];
}
```

Explanation:
Classic subset sum DP. `dp[j]` = ways to get sum j. Counting subsets summing to half the total.

---

5. Find the missing element in an unsorted array where every element appears twice except one

```
public int SingleNumber(int[] nums) {
    int result = 0;
    foreach (var num in nums) {
        result ^= num;
    }
    return result;
}
```

Explanation:
XOR of all elements cancels duplicates, leaving the single unique element.

---

6. Search for a range (find start and end indices of target in sorted array)

```
public int[] SearchRange(int[] nums, int target) {
    int left = FindBoundary(nums, target, true);
    int right = FindBoundary(nums, target, false);
    return new int[] { left, right };
}

private int FindBoundary(int[] nums, int target, bool findFirst) {
```

```
    int left = 0, right = nums.Length - 1;
    int boundary = -1;

    while (left <= right) {
        int mid = left + (right - left) / 2;
        if (nums[mid] == target) {
            boundary = mid;
            if (findFirst)
                right = mid - 1;
            else
                left = mid + 1;
        }
        else if (nums[mid] < target) left = mid + 1;
        else right = mid - 1;
    }
    return boundary;
}
```

Explanation:
Binary search twice — once to find first occurrence and once for last occurrence.

1. Find all prime factors of a number

```
public List<int> PrimeFactors(int n) {
    List<int> factors = new List<int>();

    // Print the number of 2s that divide n
    while (n % 2 == 0) {
        factors.Add(2);
        n /= 2;
    }

    // n must be odd at this point
    for (int i = 3; i * i <= n; i += 2) {
        while (n % i == 0) {
            factors.Add(i);
            n /= i;
        }
    }
```

```
    // If n is a prime number > 2
    if (n > 2) {
        factors.Add(n);
    }

    return factors;
}
```

Explanation:
We repeatedly divide by 2, then check odd factors up to √n. If leftover n > 2, it's prime.

---

2. Count the number of set bits (1s) in binary representation of an integer

```
public int CountSetBits(int n) {
    int count = 0;
    while (n != 0) {
        count += n & 1;
        n >>= 1;
    }
    return count;
}
```

Explanation:
Shift through each bit; add 1 to count if least significant bit is set.

Alternative using Brian Kernighan's algorithm:

```
public int CountSetBits(int n) {
    int count = 0;
    while (n != 0) {
        n &= (n - 1); // Drops the lowest set bit
        count++;
    }
    return count;
}
```

---

3. Calculate power of a number without built-in pow()

```
public double Power(double x, int n) {
    if (n == 0) return 1;
    double half = Power(x, n / 2);
    if (n % 2 == 0)
        return half * half;
    else
        return n > 0 ? x * half * half : (half * half) / x;
}
```

Explanation:
Uses fast exponentiation (divide and conquer) to calculate x^n in O(log n).

---

4. Check if a number is a perfect square

```
public bool IsPerfectSquare(int num) {
    if (num < 0) return false;
    int left = 0, right = num;
    while (left <= right) {
        int mid = left + (right - left) / 2;
        long sq = (long)mid * mid;
        if (sq == num) return true;
        else if (sq < num) left = mid + 1;
        else right = mid - 1;
    }
    return false;
}
```

Explanation:
Binary search for integer square root and check if square equals num.

---

5. Find gcd and lcm of two numbers

```
public int GCD(int a, int b) {
```

```
    while (b != 0) {
        int temp = b;
        b = a % b;
        a = temp;
    }
    return a;
}

public int LCM(int a, int b) {
    return (a / GCD(a, b)) * b;
}
```

Explanation:
GCD uses Euclidean algorithm. LCM calculated via `LCM(a,b) = (a*b)/GCD(a,b)`.

---

6. Find number of trailing zeroes in factorial of a number

```
public int TrailingZeroes(int n) {
    int count = 0;
    while (n > 0) {
        n /= 5;
        count += n;
    }
    return count;
}
```

Explanation:
Count factors of 5 in factorial since 2s are plentiful, trailing zeros depend on 5s.

1. Find all anagrams of a string in another string

```
public List<int> FindAnagrams(string s, string p) {
    List<int> result = new List<int>();
    if (p.Length > s.Length) return result;

    int[] pCount = new int[26];
    int[] sCount = new int[26];
```

```
    for (int i = 0; i < p.Length; i++) {
        pCount[p[i] - 'a']++;
        sCount[s[i] - 'a']++;
    }

    if (Enumerable.SequenceEqual(pCount, sCount))
        result.Add(0);

    for (int i = p.Length; i < s.Length; i++) {
        sCount[s[i] - 'a']++;
        sCount[s[i - p.Length] - 'a']--;
        if (Enumerable.SequenceEqual(pCount, sCount))
            result.Add(i - p.Length + 1);
    }

    return result;
}
```

Explanation:
Sliding window with frequency count arrays for the pattern and current window.

---

2. Find the longest common prefix among a list of strings

```
public string LongestCommonPrefix(string[] strs) {
    if (strs == null || strs.Length == 0) return "";

    for (int i = 0; i < strs[0].Length; i++) {
        char c = strs[0][i];
        for (int j = 1; j < strs.Length; j++) {
            if (i == strs[j].Length || strs[j][i] != c)
                return strs[0].Substring(0, i);
        }
    }
    return strs[0];
}
```

Explanation:
Check character by character across all strings until mismatch.

---

3. Implement an iterator for a nested list (flatten a nested list of integers)

```
public class NestedIterator {
    private Queue<int> queue;

    public NestedIterator(IList<NestedInteger> nestedList) {
        queue = new Queue<int>();
        Flatten(nestedList);
    }

    private void Flatten(IList<NestedInteger> nestedList) {
        foreach (var ni in nestedList) {
            if (ni.IsInteger()) queue.Enqueue(ni.GetInteger());
            else Flatten(ni.GetList());
        }
    }

    public bool HasNext() {
        return queue.Count > 0;
    }

    public int Next() {
        return queue.Dequeue();
    }
}
```

Note:
`NestedInteger` is an interface with methods: `IsInteger()`, `GetInteger()`, `GetList()`.

Explanation:
Pre-flatten the nested list into a queue and iterate over it.

---

4. Find the maximum sum path in a triangle (bottom-up DP)

```
public int MaximumTotal(IList<IList<int>> triangle) {
    int n = triangle.Count;
    int[] dp = new int[n];
    for (int i = 0; i < n; i++) dp[i] = triangle[n - 1][i];

    for (int layer = n - 2; layer >= 0; layer--) {
        for (int i = 0; i <= layer; i++) {
            dp[i] = triangle[layer][i] + Math.Max(dp[i], dp[i + 1]);
        }
    }
    return dp[0];
}
```

Explanation:
Start from bottom row, keep updating max path sums up to the top.

---

5. Solve the "Find the Celebrity" problem

```
public int FindCelebrity(int n, Func<int, int, bool> knows) {
    int candidate = 0;
    for (int i = 1; i < n; i++) {
        if (knows(candidate, i)) candidate = i;
    }

    for (int i = 0; i < n; i++) {
        if (i != candidate && (knows(candidate, i) || !knows(i,
candidate)))
            return -1;
    }
    return candidate;
}
```

Explanation:
First find candidate by elimination, then verify candidate.

---

6. Implement a trie (prefix tree) for string matching

```
public class TrieNode {
    public TrieNode[] Children = new TrieNode[26];
    public bool IsEnd = false;
}

public class Trie {
    private TrieNode root;

    public Trie() {
        root = new TrieNode();
    }

    public void Insert(string word) {
        TrieNode node = root;
        foreach (char c in word) {
            int idx = c - 'a';
            if (node.Children[idx] == null)
                node.Children[idx] = new TrieNode();
            node = node.Children[idx];
        }
        node.IsEnd = true;
    }

    public bool Search(string word) {
        TrieNode node = SearchNode(word);
        return node != null && node.IsEnd;
    }

    public bool StartsWith(string prefix) {
        return SearchNode(prefix) != null;
    }

    private TrieNode SearchNode(string word) {
        TrieNode node = root;
        foreach (char c in word) {
            int idx = c - 'a';
            if (node.Children[idx] == null) return null;
```

```
            node = node.Children[idx];
        }
        return node;
    }
}
```

Explanation:
Standard trie with insert, search, and prefix checking.

---

7. Solve "Find the Duplicate Number" problem in an array of n+1 integers

```
public int FindDuplicate(int[] nums) {
    int slow = nums[0], fast = nums[0];
    do {
        slow = nums[slow];
        fast = nums[nums[fast]];
    } while (slow != fast);

    fast = nums[0];
    while (slow != fast) {
        slow = nums[slow];
        fast = nums[fast];
    }
    return slow;
}
```

Explanation:
Floyd's Tortoise and Hare cycle detection, treating values as pointers.

---

8. Find the longest substring with exactly two distinct characters

```
public int LengthOfLongestSubstringTwoDistinct(string s) {
    int left = 0, right = 0, maxLen = 0;
    Dictionary<char, int> map = new Dictionary<char, int>();

    while (right < s.Length) {
```

```
            char c = s[right];
            map[c] = right;

            if (map.Count > 2) {
                int delIndex = map.Values.Min();
                map.Remove(s[delIndex]);
                left = delIndex + 1;
            }

            maxLen = Math.Max(maxLen, right - left + 1);
            right++;
        }
        return maxLen;
}
```

Explanation:
Sliding window with hashmap to track indices of distinct chars, remove the leftmost when >2.

---

9. Check if a string has balanced brackets

```
public bool IsBalanced(string s) {
    Stack<char> stack = new Stack<char>();
    Dictionary<char, char> pairs = new Dictionary<char, char> {
        {')', '('}, {']', '['}, {'}', '{'}
    };

    foreach (char c in s) {
        if ("([{".Contains(c))
            stack.Push(c);
        else if (")]}".Contains(c)) {
            if (stack.Count == 0 || stack.Pop() != pairs[c])
                return false;
        }
    }
    return stack.Count == 0;
}
```

Explanation:
Use a stack to match opening and closing brackets properly.

---

10. Rotate a matrix by 90 degrees in place

```
public void RotateMatrix(int[][] matrix) {
    int n = matrix.Length;
    // Transpose
    for (int i = 0; i < n; i++) {
        for (int j = i + 1; j < n; j++) {
            int temp = matrix[i][j];
            matrix[i][j] = matrix[j][i];
            matrix[j][i] = temp;
        }
    }
    // Reverse each row
    for (int i = 0; i < n; i++) {
        Array.Reverse(matrix[i]);
    }
}
```

Explanation:
Transpose matrix and then reverse each row to rotate 90° clockwise.

1. Find the single non-repeated element in an array where every other element appears twice

```
public int SingleNonRepeated(int[] nums) {
    int result = 0;
    foreach (int num in nums) {
        result ^= num;
    }
    return result;
}
```

Explanation:
XOR of a number with itself is 0; XOR with 0 is the number. So duplicates cancel out, leaving the unique number.

---

2. Count the number of 1s in the binary representation of a number

```
public int CountOnes(int n) {
    int count = 0;
    while (n != 0) {
        n &= (n - 1);  // Drops the lowest set bit
        count++;
    }
    return count;
}
```

Explanation:
Brian Kernighan's algorithm efficiently removes the lowest set bit each iteration until zero.

---

3. Check if two numbers have opposite signs

```
public bool HaveOppositeSigns(int x, int y) {
    return (x ^ y) < 0;
}
```

Explanation:
XOR of two numbers with opposite signs has the sign bit set (negative number).

---

4. Find the missing number in a sequence from 1 to n using XOR

```
public int FindMissingNumber(int[] nums, int n) {
    int xor = 0;
    for (int i = 1; i <= n; i++) {
        xor ^= i;
    }
    foreach (int num in nums) {
        xor ^= num;
    }
    return xor;
}
```

Explanation:
XOR all numbers from 1 to n and XOR all elements in array; duplicates cancel out, leaving missing number.

---

5. Swap two numbers without using a temporary variable

```
public void Swap(ref int a, ref int b) {
    if (a != b) {
        a ^= b;
        b ^= a;
        a ^= b;
    }
}
```

Explanation:
XOR swap algorithm exchanges values without extra storage. (Check `a != b` to avoid zeroing when both are same.)

---

6. Reverse the bits of a number

```
public uint ReverseBits(uint n) {
    uint result = 0;
    for (int i = 0; i < 32; i++) {
        result <<= 1;
        result |= (n & 1);
        n >>= 1;
    }
    return result;
}
```

Explanation:
Iteratively take least significant bit of n, add it to result's LSB, shift both accordingly.